
GeoGenBench: A Benchmark for Generative Geometric Reasoning with Symbolic Verification

Anonymous Author(s)

Affiliation

Address

email

Abstract

Existing geometry benchmarks ask language models to *interpret* a diagram and answer a question. None ask whether a model can *produce* a correct diagram from a description. We introduce **GeoGen**, a methodology for automatically-verifiable benchmarks of generative geometric reasoning, and **GeoGenBench**, its first instance: 801 prompts paired with machine-checkable property lists (e.g. `right_angle`, `midpoint`, `parallel`; 17 predicate types in total). **GEOGEN** co-generates prompts and rubrics from shared Python source (eliminating drift between question and grading criteria), then renders the model’s TikZ output and decides each predicate symbolically against compiled SymPy coordinates — no human grading, no vision–language judge.

We benchmark six frontier models from three vendors under two strategies: *raw TikZ* (the model emits TikZ directly) and a structured *intermediate representation* (IR; the model emits typed JSON declaring points, lines, intersections, and self-checks, which we compile to TikZ before rendering). **The cost–quality Pareto frontier collapses to a single corner.** Under the IR strategy, GPT-5.5 (94.1%, \$0.075/scen), Claude Sonnet 4.6 (93.4%, \$0.088), and Claude Opus 4.7 (93.3%, \$0.532) tie within 1 p.p. at a $7\times$ cost spread. On raw TikZ, GPT-5.5 (91.9%, \$0.058) beats Claude Opus 4.7 (71.6%, \$0.262) by 20 p.p. at $4.5\times$ lower cost. The IR adds 2–25 p.p. over raw TikZ, largest for the models weakest at raw. One vendor (Gemini Flash) rejects the IR as exceeding its structured-output complexity cap — a portability gap, not a capability gap. A pre-registered $N=200$ human-correlation study finds moderate auto/expert agreement ($\kappa=0.455$) with one-directional verifier lenience: $\sim 18\%$ of **pass** records overturn to expert **fail**, zero the reverse, concentrated on templates whose property lists do not enforce prompt-specified numerics. We release the dataset, the template engine, the verification toolkit, and a public leaderboard.

1 Introduction

Modern frontier LLMs author entire visual artifacts: slide decks, worksheets, technical illustrations, engineering diagrams. The most-deployed visual artifact in the K–12 educational market is the geometry diagram — a 2D plane figure paired with a problem statement. Yet every existing geometry benchmark we are aware of evaluates the inverse direction: given a

34 diagram, answer a question about it. None measures whether a model can *produce* a correct
 35 diagram from a textual description. This is the gap GEOGENBENCH fills.

36 **Motivating example.** “Draw an acute triangle EFG with vertex angles 60° , 70° at E
 37 and F . Construct the altitude from G meeting EF at H .” A correct rendering satisfies:
 38 $\angle E = 60^\circ$, $\angle F = 70^\circ$, H on segment EF (not its extension), $\angle GHE = 90^\circ$, triangle acute.
 39 GEOGENBENCH encodes each clause as a machine-checkable predicate against the SymPy-
 40 compiled coordinates of the model’s rendered TikZ output: in our pilot, Claude Haiku 4.5 and
 41 Claude Opus 4.7 pass under both strategies; GPT-5.5 + `raw_code` places H on the line EF
 42 but *outside* the segment; Claude Sonnet 4.6 + `raw_code` draws an obtuse triangle. The verifier
 43 flags each failure with a specific predicate verdict, not a holistic score (Figure 1 illustrates
 44 the prompt→rubric pipeline; the rendered-then-verified pipeline is in Section 4.1).

45 **Methodology vs. instance.** We use **GeoGen** for the methodology: (i) procedural
 46 co-generation of prompts and property-list rubrics from shared source code, and (ii) a
 47 render-then-symbolic-verify pipeline driven by a declarative predicate language (Section 3).
 48 **GeoGenBench** is the 801-prompt plane-geometry instance. The distinction matters:
 49 future GEOGEN-family benchmarks for coordinate-plane algebra, 3D solids, and engineering
 50 drawings would inherit the methodology while extending the per-domain symbolic primitives.

51 Contributions.

- 52 1. **The GeoGen methodology and reusable verification toolkit** — procedural
 53 co-generation eliminating prompt/rubric drift; render-then-symbolic-verify with a
 54 versioned 17-predicate declarative language; a hardening audit (Section 4.1) that
 55 closed 4.6% of silently-skipped pilot checks across three passes. We release the
 56 engine, verifier, predicate language v1, and Docker renderer as reusable code.
- 57 2. **The GeoGenBench instance** — 801 prompts (600 procedurally co-generated,
 58 201 LLM-extracted from a curriculum scaffold), six frontier models from three
 59 vendors $\times$ two strategies, on a 600-scenario $\times$ 3-repeat draw. Headline findings: a
 60 strict-pass spread of 58 p.p. ($35.9\% \rightarrow 94.1\%$) across eleven measurable cells; a $7\times$
 61 cost spread at equivalent quality between GPT-5.5 + `structured` (94.1%, \$0.075/scen)
 62 and Claude Opus 4.7 + `structured` (93.3%, \$0.532/scen); and a portability finding
 63 (Section 6.3: Gemini 2.5 Flash’s schema-state cap blocks structured generation). A pre-
 64 registered $N=200$ human-correlation study (Section 4.2) finds moderate auto/expert
 65 agreement ($\kappa=0.455$) with one-directional verifier lenience concentrated on the loose-
 66 completeness templates we identify in Section 4.1. The dataset, template engine,
 67 verification toolkit, and leaderboard data are all released under permissive licenses.

68 **Headline empirical results.** The structured strategy lifts every model by +2.2 to
 69 +24.7 p.p.: GPT-5.5 + 2.2, Claude Haiku 4.5 + 4.8, Claude Sonnet 4.6 + 12.2, Claude Opus 4.7
 70 + 21.7, Gemini 2.5 Pro + 24.7 p.p.. Models strong at raw TikZ (GPT-5.5) gain little from scaf-
 71 folding; models weak at raw (Claude Opus 4.7, Gemini 2.5 Pro) gain the most. Under structured,
 72 GPT-5.5 (94.1%, \$0.075/scen), Claude Sonnet 4.6 (93.4%, \$0.088), and Claude Opus 4.7 (93.3%,
 73 \$0.532) span only 0.8 p.p. at a $7\times$ cost ratio. Claude Opus 4.7 + `raw_code` (\$0.262/scen,
 74 71.6%) is the most decisively dominated cell: GPT-5.5 + `raw_code` (\$0.058, 91.9%) gives
 75 20 p.p. better pass at $4.5\times$ lower cost. The Pareto frontier collapses such that all three
 76 frontier+IR cells co-locate.

77 2 Related work

78 We organise prior work along two axes: (i) whether the model is asked to *interpret* a diagram
 79 or *produce* one, and (ii) whether grading is symbolic, vision-language, or fixed-answer. Table 1
 80 positions GEOGENBENCH against the closest prior benchmarks.

81 **Geometry QA benchmarks.** Geometry3K [7], GeoQA [3], UniGeo [4], MathVista [8],
 82 GeoEval [11], and GeoBench [1] all measure whether a model can *interpret* a given diagram
 83 and answer a question about it. GEOGENBENCH instead measures whether a model can

Table 1: GEOGENBENCH vs. closest prior benchmarks. *Auto-grade*: grading does not require human or MLLM judgment.

Benchmark	Year	Task	# items	Auto-grade	Modality
Geometry3K [7]	2021	QA	3,002	yes (MC)	image+text
GeoQA [3]	2021	QA (program)	4,998	yes (exec)	image+text
UniGeo [4]	2022	QA + proving	14,541	yes (seq)	image+text
MathVista [8]	2024	QA (mixed)	6,141	partial	image+text
GeoEval [11]	2024	QA + variants	5,050	yes	image+text
GeoBench [1]	2026	hierarchical QA	1,021	yes (verified)	image+text
T2I-CompBench++ [6]	2025	T2I gen	8,000	partial (VLM)	text $\rightarrow$ image
GenExam [10]	2025	T2I exam gen	1,000	LLM-judge	text $\rightarrow$ image
GeoGenBench (ours)	2026	diagram gen	801	yes (symbolic)	text $\rightarrow$ TikZ $\rightarrow$ SVG

84 *produce* a correct diagram from a textual description — the deployed task when LLMs author
85 worksheets, slides, and technical illustrations.

86 **Generation-side benchmarks.** T2I-CompBench++ [6] and GenExam [10] grade gener-
87 ated images via vision-language judges (CLIP, BLIP, MLLMs) or LLM judges; these cannot
88 tell whether a “right triangle” actually has a right angle, only whether the image *looks like* one.
89 GEOGENBENCH grades in symbolic space (TikZ $\rightarrow$ compiled SymPy geometry) where exact
90 verification is possible: a `right_angle_at_C` verdict is `Triangle(B, C, A).angles[C] ==`
91 `$\pi/2 \pm \tau$` , not a judge’s opinion. GenExam is the closest prior art (generative-exam framing);
92 we differ on grading mechanism (deterministic SymPy vs MLLM judge, $\sim 1000\times$ cheaper
93 and fully reproducible) and modality (TikZ vs raster).

94 **Symbolic / formal grading.** Lean and Isabelle corpora (miniF2F, ProofNet) and Alpha-
95 Geometry’s neuro-symbolic deduction engine [9] apply machine-checked grading to *proofs*;
96 GEOGENBENCH extends the automatic-verification tradition into *diagram generation*, where
97 the verifiability target is a geometric configuration rather than a propositional proof. GE-
98 OGENBENCH occupies the previously-empty cell of *generation-side, automatically-graded*
99 geometry, enabled by the choice to grade in symbolic space rather than pixel space.

100 3 Methodology and instance

101 GEOGEN is a methodology for building automatically-verifiable benchmarks of generative
102 geometric reasoning; GEOGENBENCH is its first instantiation, for plane Euclidean geometry.
103 The methodology rests on two pillars.

104 **Pillar 1: Procedural co-generation of prompts and rubrics.** Both the natural-
105 language prompt and the machine-checkable property list are emitted from the same Python
106 source code (Figure 1). A single `altitude_template(A, B, C, H)` function returns both
107 a prose request to draw the altitude and the predicate list [`right_angle(A, H, B)`,
108 `point_on_segment(H, B, C)`] that any correct rendering must satisfy. This eliminates
109 rubric/question drift and provides a contamination defence (reviewers can re-roll the engine
110 with new seeds for fresh splits at the same difficulty distribution). For domains where
111 procedural co-generation is impractical (3D, multi-step proofs), the methodology supports a
112 fallback mode: hand- or LLM-extracted prompts paired with hand-authored property lists,
113 with metrics reported separately by source.

114 **Pillar 2: Render-then-symbolic-verify with a declarative predicate language.** The
115 model emits TikZ; a containerised LuaLaTeX renderer compiles to SVG; an extractor compiles
116 the same source to a coordinate-bearing SymPy representation; a fixed-vocabulary, typed
117 predicate language (Table 3) is decided against the SymPy representation in microseconds
118 per predicate. The language is *declarative* (prompt authors specify *what* must be true, not
119 *how* to check it), *versioned* (Section B formalises v1: 17 symbolic + 2 source-string predicates;
120 future instantiations extend without disturbing existing benchmarks), and *renderer-agnostic*

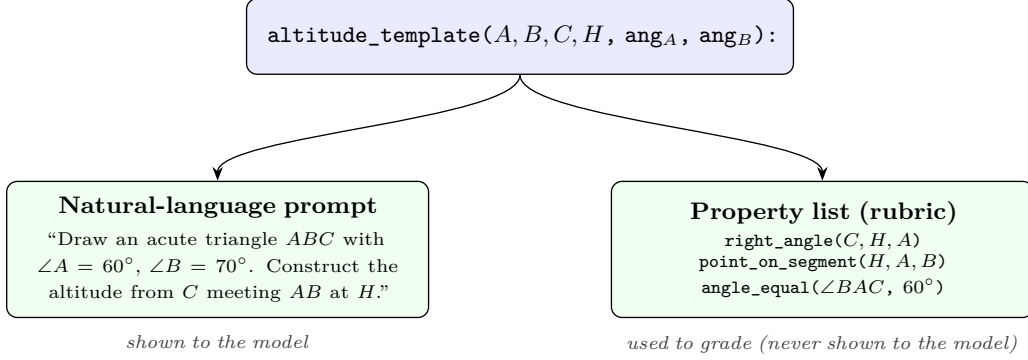


Figure 1: Procedural co-generation: one Python template emits both the prompt and the rubric, guaranteeing they stay mutually consistent.

Table 2: GEOGENBENCH dataset composition.

Split	#scen.	T1	T2	T3	Avg. #props	Constructions
Templated	600	200	200	200	3.2	30 templates
Curriculum	201	36	128	37	6.0	73 unique tags
Total	801	236	328	237	—	100+

(we ship an alternate SVG-output strategy alongside TikZ to demonstrate the swap is a one-paragraph change).

3.1 Task formulation

A GEOGENBENCH scenario is $\mathcal{S} = \langle \text{prompt}, \mathcal{P}, \mathcal{L}, \mathcal{C} \rangle$: a natural-language drawing request, a list of expected geometric properties $\mathcal{P}$, required labels $\mathcal{L}$, and optional structural checks $\mathcal{C}$. A model produces a TikZ source string; we (1) render via LuaLaTeX, (2) verify by compiling the diagram’s coordinate set and applying each property in $\mathcal{P}$ against a SymPy symbol table, (3) score as one of **pass** (all checks passed), **soft_pass** (no failures but ≥ 1 check skipped), **fail**, or **gen_failure/render_failure**. The model is *not* given the property list during generation — properties are the rubric, not the input.

3.2 Scenario sources

Table 2 summarises the benchmark composition. **Templated scenarios** (600, 200 per tier) are emitted by 30 procedural templates (full enumeration in Section A) covering single shapes (T1), shapes with one auxiliary construction (T2), and multi-step compositions (T3); property-prompt consistency is by construction. **Curriculum scenarios** (201, distributed 36/128/37 across tiers) are LLM-extracted (Claude Sonnet 4.6) prompts authored against the topical scaffold of the Carnegie Learning *Bluebonnet Learning* K–12 mathematics curriculum [2] adopted by the Texas Education Agency. We use Bluebonnet only for chapter / lesson topics — no textbook prose is reproduced; only LLM-generated prompt rewrites + author-written property rubrics are released, with per-scenario lineage tags for traceability. This adds breadth (3D solids, transformations, coordinate-overlay constructions) at the cost of LLM-authored ground truth that is spot-checkable but not guaranteed correct in the same way templated GT is. We report metrics decomposed by source so reviewers can read the templated half as a calibration anchor.

3.3 Property language and tiers

The v1 vocabulary is 15 symbolic predicate types implemented in the eval verifier, plus 2 source-string check types (label and mark presence; intentionally weaker, contributing to **soft_pass** only). A further 3 predicates (**opposite_side**, **same_side**, **not_between**) are

Table 3: The GEOGEN predicate language v1. Symbolic predicates are exact checks against compiled coordinates; source-string checks contribute only to *loose* pass. Three reserved predicates are documented in Section B.3.

Predicate	Args	Decision
<i>Symbolic (exact, implemented in v1)</i>		
<code>right_angle</code>	$[a, o, b]$	$ \vec{oa} \cdot \vec{ob} < \tau$
<code>midpoint</code>	$[m, a, b]$	$\max(m_x - (a_x + b_x)/2 , m_y - (a_y + b_y)/2) < \tau$
<code>centroid</code>	$[g, a, b, c]$	componentwise centroid formula $< \tau$
<code>collinear, parallel, perpendicular</code>	—	cross/dot product $< \tau$
<code>equal_lengths</code>	$[[a, b], \dots]$	all pairwise $ AB - CD < \tau$
<code>point_on_line/segment/circle</code>	$[p, \dots]$	incidence within τ
<code>tangent, angle_bisector, angle_equal</code>	—	composite checks
<code>intersects, equidistant_from_sides</code>	—	intersection / side-distance
<i>Source-string (soft, contribute only to <code>soft_pass</code>)</i>		
<code>label_present, mark_present</code>	—	TikZ primitive appears in source

reserved for v1.1 negative-incidence templates. Each predicate is decided in microseconds against compiled coordinates with numeric tolerance $\tau = 5 \times 10^{-3}$. Table 3 summarises the language; Section B gives the full signatures, semantics, and code references. Tier labels at template-definition time: T1 (single shape, no auxiliary construction), T2 (one shape + one auxiliary), T3 (multi-step / composed).

4 Verification reliability

This section defines what verification means in GEOGENBENCH, audits the verifier for completeness and adversarial soundness, and reports the headline result of a pre-registered $N=200$ human-correlation study. Full per-claim methodology and supporting analyses are in Sections C and E.

4.1 Completeness, soundness, hardening

A property list $\mathcal{P}$ is *complete* for scenario $\mathcal{S}$ if every diagram passing all of $\mathcal{P}$ is congruent (up to rigid motion + scale) to the intended construction. We classify each templated scenario along a **completeness spectrum**: **tight** (uniquely determines the figure), **loose** (named relations pinned, continuous parameters free; intentional, matching educational practice), **skeletal** (only incidence). The audit yields 18/11/1 tight / loose / skeletal templates, giving an aggregate 63% tight / 35% loose / 2% skeletal at the scenario level (379 / 209 / 12 of 600); per-scenario witnesses for 25 tight + 25 loose are in Section C.

Adversarial soundness. For each of the 17 implemented symbolic predicates we attempt to construct a TikZ output that passes the predicate but contradicts the prompt. Five predicates show measurable soft spots — `midpoint`, `point_on_segment` (pre-hardening), `parallel`, `tangent`, and `equidistant_from_sides` — all addressable by hardening or template-side pairing (full table in Section E adversarial subsection). Constructed adversarial examples pass the verifier; we estimate $< 5\%$ of strict passes are vulnerable to such constructions in the wild because templates pair check types in ways that close single-check loopholes (e.g. midpoint scenarios always include the segment context that makes the perpendicular-bisector escape unphysical).

Hardening pass. Three changes to the verifier addressing pilot-data soft spots: (a) closed-form solver for `\tkzInterCC` / `\tkzInterLC` runtime intersections; (b) a pgfmath-style coordinate evaluator (AST walker over `\pgfmathsetmacro` chains); (c) the `centroid` predicate (added; pre-hardening it emitted no decision on `tpl-t3-cen`). Aggregate impact on the pilot: 23 of 498 records (4.6%) promote `soft_pass` $\rightarrow$ `pass`; 0 demotions. The strategy-effect (structured vs raw_code) shrinks from +10–30 p.p. pre-hardening to +4–20 p.p. after,

182 indicating about half of the prior gap was verifier weakness on `raw_code` outputs, not model
 183 weakness. Section 6 reports post-hardening numbers as canonical.

184 4.2 Human-correlation study ($N=200$, two domain-expert raters)

185 We pre-register and run a human-correlation study against the canonical post-hardening
 186 pilot ($N=498$); the pre-registration is committed to `docs/human_study_protocol.md` prior
 187 to data collection, with full methodology in Section E. The deterministic stratified sample
 188 (seed 42, restricted to `raw_code/structured`; deviation D-1) takes all 28 `soft_pass` records,
 189 all 21 rendered-but-failed records, plus 151 `pass` records balanced across (tier $\times$ strategy $\times$
 190 model). Two domain-expert raters (PhD mathematician + K-12 mathematics educator, both
 191 authors — mitigated by Q1 blinding and per-rater κ reporting) annotate independently. **Q1**
 192 (**blind**): “Does the diagram correctly satisfy every claim in the prompt?” {yes / partially /
 193 no}; the rating tool reveals neither the auto-verdict nor any per-predicate values until Q1 is
 194 submitted. **Q2 (post-Q1)**: per-predicate {agrees-pass / agrees-fail / disagrees / unsure}.
 195 After independent annotation, raters meet (≤ 90 minutes) to discuss Q1 disagreements to
 196 convergence, producing one consensus label per item. The pre-registered primary endpoint
 197 is $\kappa(\text{auto}, \text{consensus}) \geq 0.70$ on the binary collapse, with the bootstrap-CI lower bound also
 198 exceeding 0.70.

199 **Headline result.** $\kappa = 0.455$ on the binary collapse, $N=200$, 95% bootstrap CI
 200 $[0.304, 0.590]$ over 1000 resamples; balanced accuracy 0.684. The point estimate falls
 201 in the protocol’s middle response-tree branch ($0.40 \leq \kappa < 0.70$), so we report the result as
 202 **moderate agreement**, with the lower bound below the pre-registered 0.70 threshold (the
 203 headline-validation hypothesis is not supported). Inter-rater agreement before consensus is
 204 substantial ($\kappa = 0.704$), so the rating instrument is consistent — the gap is between the two
 205 raters as a unit and the auto-verdict, not between the raters.

206 **Interpretation: one-directional lenience.** The disagreement is one-directional: 36
 207 of 200 records have the verifier accepting where humans reject (18% of strict-pass records
 208 overturn to **fail** on consensus review), and *zero* records have the reverse. The verifier
 209 is systematically lenient relative to expert humans, never strict. Per-tier κ : $T1 = 0.651$
 210 (substantial), $T2 = 0.160$ (slight), $T3 = 0.796$ (substantial); the entire disagreement
 211 concentrates on T2, where the property list checks the named auxiliary construction (altitude
 212 perpendicular, median to midpoint) but not the prompt’s specific numeric angle / side
 213 parameters. This is the loose-completeness pattern (Section 4.1) made empirical: in the
 214 qualitative coding (Section E.7), 32/36 disagreements (89%) code as “verifier missed prompt-
 215 numeric constraint.”

216 **Implications for the leaderboard.** The headline pass-rates in Section 6 should be read
 217 with one caveat: the auto-verdict is one-directionally lenient relative to expert humans,
 218 with the lenience concentrated on T2 auxiliary-construction templates whose property lists
 219 do not enforce prompt-specified numerics. Concretely, the strict pass-rates in Table 4 are
 220 *conservative against the verifier-bias axis* (true expert-rated rates would be lower, not inflated
 221 by spurious fails), and rate *differences* between cells are largely preserved because the lenience
 222 applies uniformly across (model, strategy) cells on the affected templates. GEOGENBENCH
 223 pass-rate is a moderately-but-not-perfectly-correlated proxy for expert agreement, with the
 224 gap concretely localised and the v2 fix (template-side numeric predicates lifting affected T2
 225 templates from loose to tight) committed as follow-up work.

226 5 Models and strategies

227 **Models.** We evaluate six production frontier models spanning $\sim 20\times$ in price across three
 228 vendors: Claude Opus 4.7 (\$15/\$75 per M input/output tokens), Claude Sonnet 4.6 (\$3/\$15),
 229 Claude Haiku 4.5 (\$1/\$5), GPT-5.5 (\$5/\$20), Gemini 2.5 Pro (\$1.25/\$10), and Gemini 2.5 Flash
 230 (\$0.30/\$2.50). All runs use temperature 1.0; costs use each model’s published per-token rate
 231 as of April 2026.

Common prompt: “Draw an acute triangle EFG with $\angle E = 60^\circ$, $\angle F = 70^\circ$. Construct the altitude from G meeting EF at H .”

raw_code <i>zero scaffolding — LLM emits TikZ directly</i>	structured <i>schema-constrained IR — LLM emits typed JSON, we compile to TikZ</i>
<pre> \coordinate (E) at (0,0); \coordinate (F) at (5,0); \coordinate (G) at (3.07,5.31); \coordinate (H) at (3.07,0); \draw (E)--(F)--(G)--cycle; \draw[densely dashed] (G) -- (H); </pre>	<pre> { "points": [{ "name": "E", "type": "free" }, { "name": "F", "type": "free" }, { "name": "G", "type": "by_angles", "from": "E", "to": "F", "ang_at_from": 60, "ang_at_to": 70 }, { "name": "H", "type": "intersection", "lines": [{"G", "perp_EF"}, {"E", "F"}] }, { "name": "check", "type": "right_angle", "args": ["G", "H", "E"]}] } </pre>

Figure 2: The two reference strategies on the same prompt. Both converge to the same TikZ→SVG→SymPy→predicate-verdict pipeline.

Strategies. GEOGENBENCH ships two reference strategies on a spectrum from “maximum LLM freedom” to “schema-constrained intermediate representation”; both share a common **SubstanceStrategy** interface and either can be replaced by any text-emitting model. Figure 2 shows what the LLM emits in each on the same prompt. **raw_code:** the LLM is given the prompt plus a TikZ tutorial system prompt and produces TikZ directly via a `render_diagram(tikz: str)` tool call. It may revise based on the rendered output but no symbolic checks are run. This is the zero-scaffolding baseline. **structured:** the LLM is given a Pydantic schema for a typed intermediate representation **DiagramIR** (point declarations, segments, intersections, circles, render ops, *check* declarations) and emits IR JSON; *code* (not the LLM) compiles $\text{IR} \rightarrow \text{SymPy} \rightarrow \text{TikZ} \rightarrow \text{SVG}$. The IR’s check list runs *before* render, so the strategy self-corrects: if a declared check fails, the strategy revises (up to $N=3$ retries). This is the strategy we recommend for production deployment. The codebase additionally ships `strategies/raw_svg.py` (text $\rightarrow$ SVG, no TeX); we exclude it from the headline since it measures essentially the same task as raw TikZ.

Cost and latency. For each (model, strategy, scenario, repeat) we record (a) total input/output tokens including retries, (b) wall-clock duration from prompt submission to rendered SVG, and (c) the token-multiplied USD cost. The *mean cost per scenario* reported in Section 6 is the arithmetic mean across the headline run including failed scenarios. The headline run draws each of the 600 templated scenarios with three repeats per cell, with up to three internal IR-check retries on **structured**.

6 Results

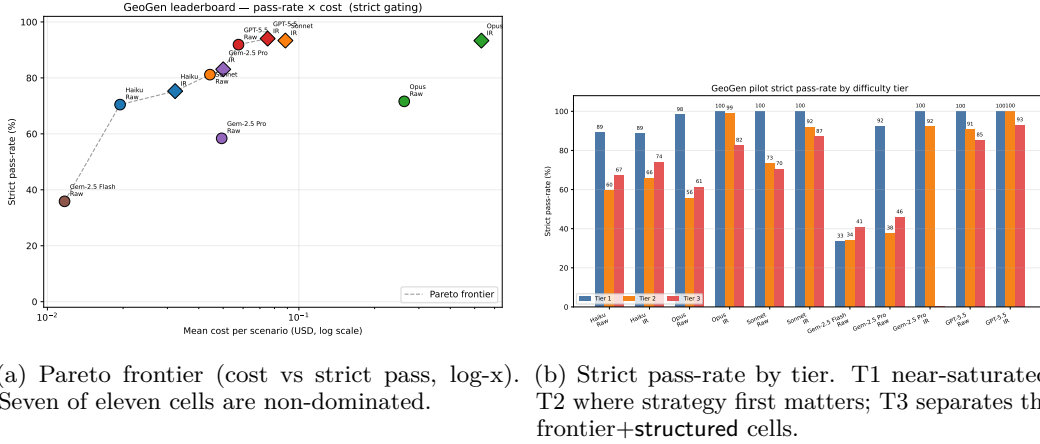
Setup. We evaluate the 12 (model $\times$ strategy) cells at 600 scenarios $\times$ 3 repeats each. A subprocess-isolated runner with an OS-level kill timeout caught 62 pathology scenarios across the **structured** cells (LLM-generated IRs that trigger circular-dependency cycles), lifting per-cell coverage to $\geq 91\%$ for every cell except Gemini 2.5 Flash + **structured** (blocked at the API layer; see Section 6.3). Two cells are marginally partial: Claude Opus 4.7 + **structured** (574/600) and GPT-5.5 + **structured** (545/600); marked * in Table 4. The hardening passes (Section 4.1) lifted **raw_code** pilot pass-rates by +10 to +17 p.p. at pilot N, with no effect on **structured** rows; the headline run uses the post-hardened verifier throughout.

6.1 Two findings

Strategy choice helps unevenly. **structured** lifts every model: GPT-5.5 + 2.2 p.p. (91.9% $\rightarrow$ 94.1%), Claude Haiku 4.5 + 4.8, Claude Sonnet 4.6 + 12.2, Claude Opus 4.7 + 21.7,

Table 4: Headline leaderboard on the 600-scenario $\times$ 3-repeat draw (post-hardening). Strict = pass; loose = pass+soft_pass. Cells * are marginally partial. The $\dagger$ cell is blocked at the API layer by Google’s schema-state cap (Section 6.3).

Model	Strategy	N	Strict	Loose	\$/scen.	Latency
GPT-5.5	structured	2,304*	94.1%	94.1%	\$0.075	33.6 s
Claude Sonnet 4.6	structured	2,861	93.4%	93.4%	\$0.088	19.0 s
Claude Opus 4.7	structured	2,292*	93.3%	93.3%	\$0.532	23.9 s
GPT-5.5	raw_code	1,800	91.9%	95.3%	\$0.058	14.8 s
Gemini 2.5 Pro	structured	1,140*	83.1%	83.1%	\$0.050	75.7 s
Claude Sonnet 4.6	raw_code	1,800	81.2%	95.3%	\$0.044	16.4 s
Claude Haiku 4.5	structured	2,232	75.3%	76.8%	\$0.032	8.0 s
Claude Opus 4.7	raw_code	1,800	71.6%	95.3%	\$0.262	13.8 s
Claude Haiku 4.5	raw_code	2,403	70.5%	76.0%	\$0.019	9.5 s
Gemini 2.5 Pro	raw_code	1,800	58.4%	89.9%	\$0.049	32.9 s
Gemini 2.5 Flash	raw_code	2,403	35.9%	42.2%	\$0.012	22.1 s
Gemini 2.5 Flash	structured	0 $\dagger$	—	—	—	—



(a) Pareto frontier (cost vs strict pass, log-x). Seven of eleven cells are non-dominated. (b) Strict pass-rate by tier. T1 near-saturated; T2 where strategy first matters; T3 separates the frontier+structured cells.

Figure 3: Cost-quality and tier-stratified views of the headline run.

264 Gemini 2.5 Pro + 24.7. The lift is largest for the models weakest at raw_code (Claude Opus 4.7,
265 Gemini 2.5 Pro) and smallest for the strongest (GPT-5.5).

266 **Frontier convergence: three flagships tie within 1 p.p. at structured.** GPT-5.5
267 + structured (94.1%, \$0.075/scen), Claude Sonnet 4.6 + structured (93.4%, \$0.088), and
268 Claude Opus 4.7 + structured (93.3%, \$0.532) span only 0.8 p.p. at a 7 $\times$ cost ratio — the
269 frontier-tier model carries no measurable quality premium under IR scaffolding. At raw_code,
270 the cross-vendor ranking is GPT-5.5 (91.9%) > Claude Sonnet 4.6 (81.2%) > Claude Opus 4.7
271 (71.6%) $\approx$ Claude Haiku 4.5 (70.5%) > Gemini 2.5 Pro (58.4%) > Gemini 2.5 Flash (35.9%);
272 GPT-5.5 + raw_code dominates Claude Opus 4.7 + raw_code by 20 p.p. at 4.5 $\times$ lower cost.

273 6.2 Pareto and tier-stratified views

274 GPT-5.5 + structured is the unique high-quality corner (Figure 3a): it Pareto-dominates
275 Claude Sonnet 4.6 + structured (+0.8 p.p. at 17% lower cost) and Claude Opus 4.7 + structured
276 (+0.8 p.p. at 7 $\times$ lower cost). Claude Opus 4.7 + raw_code is the most-dominated cell
277 (GPT-5.5 + raw_code beats it by 20 p.p. at 4.5 $\times$ lower cost); Claude Haiku 4.5 + raw_code
278 is statistically tied with Claude Opus 4.7 + raw_code at 14 $\times$ lower cost. Gemini 2.5 Flash
279 + raw_code (\$0.012, 35.9%) is the price-floor frontier point; Gemini 2.5 Pro + structured
280 (\$0.050, 83.1%) is on the frontier with caveat (83.1% measured on T1–T2 only; T3 was
281 throttled by Google’s quota). The tier breakdown (Figure 3b): T1 is at 100% for every
282 Anthropic/OpenAI structured cell — a signal that v2 needs harder T1 templates; T2 is

where strategy first matters, with Claude Opus 4.7 jumping +43 p.p. (56% $\rightarrow$ 99%) and Gemini 2.5 Pro +54 p.p. (38% $\rightarrow$ 92%); T3 separates the three frontier+structured cells (GPT-5.5 93%, Claude Sonnet 4.6 87%, Claude Opus 4.7 82%).

Per-template heatmap and failure modes. The per-template heatmap (Figure 4, Section A) exposes **universally easy** (every T1 template at or near 100% for each model’s best strategy), **universally hard** (tp1-t3-trv at $\leq$ 16% for every cell), and **discriminating** templates (tp1-t2-bis, tp1-t2-ic, tp1-t3-2alt: model matters more than strategy). The failure-mode breakdown (Figure 5, Section F) shows failures dominated by geometric-predicate violations on the hardest templates: **corresponding_angles_equal** alone accounts for 34–48 failures per cell on tp1-t3-trv; structured failures tilt toward timeout. *Most wrong outputs are rendered diagrams with the wrong geometry*, not broken LaTeX.

6.3 Deployment finding: Gemini’s schema-state cap

The structured strategy depends on schema-constrained generation; OpenAI and Google compile the schema to a server-side FSM with vendor-specific caps. Our **DiagramIR** schema (a discriminated union of ~ 20 `DefStmt` variants) compiles to thousands of FSM states; OpenAI and Gemini 2.5 Pro accommodate this, Gemini 2.5 Flash does not (HTTP 400 in 1.1–1.6s with “the specified schema produces a constraint that has too many states for serving”). This is not a capability gap (no token is generated); it is a cross-vendor portability concern for any benchmark shipping a multi-variant discriminated-union IR. We report Gemini 2.5 Flash + `raw_code` at 35.9% and exclude the blocked structured cell. Per-template case studies (tp1-t3-trv universal failure, tp1-t3-tan strategy discriminator with -86 p.p. `raw_code` $\rightarrow$ structured gap on Claude Opus 4.7, tp1-t3-cen self-referential pass) and the per-cell stability index (GPT-5.5 + structured at 99.8% all-same-verdict; the price-floor cells in the 30–70% band) are in Section G.

7 Discussion and conclusion

Scope and limitations. GEOGENBENCH measures whether a model can translate a textual specification of a 2D plane-geometry construction into a globally-consistent metric realisation; it does not test proof construction, 3D solids, coordinate-plane algebra, or *teaching utility*. 35% of templated scenarios are loose: a passing verdict means “produced a diagram consistent with the prompt’s stated constraints,” not *the* canonical diagram. Two structured cells are marginally partial (Claude Opus 4.7 574/600, GPT-5.5 545/600) due to LLM-IR circular-dependency cycles — conservative against those pathologies. On the three cells run on curriculum at headline N, Claude Haiku 4.5 + `raw_code` drops 82.1% $\rightarrow$ 36.0%, Claude Haiku 4.5 + structured 84.5% $\rightarrow$ 36.8%, Gemini 2.5 Flash + `raw_code` 39.4% $\rightarrow$ 25.4%: the curriculum split is materially harder for every cell measured (topical breadth and LLM-authored predicate lists both contribute). Headline-run findings are templated-split findings.

Threats to validity. The procedural template engine ships with the benchmark, allowing reviewers to roll fresh contamination-free splits. The adversarial probe found 5 predicate soft spots all addressable by hardening or template-side mitigation, yielding $< 5\%$ vulnerability to constructed adversarial examples; the human-correlation study (Section 4.2) found a larger $\sim 18\%$ rate of *naturally-occurring* model outputs the verifier accepts but expert raters reject, concentrated on auxiliary-construction templates whose property lists do not enforce prompt-specified numerics.

Deployment and conclusion. For raw TikZ, GPT-5.5 + `raw_code` dominates (beats Claude Opus 4.7 + `raw_code` by 20 p.p. at $4.5\times$ lower cost; Claude Haiku 4.5 + `raw_code` tied with Claude Opus 4.7 + `raw_code` at $14\times$ lower cost). At the high-quality end, the three frontier+structured cells converge within 1 p.p. at $7\times$ cost spread; GPT-5.5 + structured is the unique Pareto corner. GEOGEN’s two pillars (Section 3) extend beyond plane geometry: procedural co-generation reuses wherever a domain admits a Python-expressible construction, and render-then-symbolic-verify extends along the predicate-language axis

(versioned, additive). Two extensions are in flight: a GenExam-Math integration [10] re-encoding a 91-prompt geometry subset for empirical comparison between automatic verification and vision-LM judges, and a coordinate-plane / 3D-solid extension. We release the dataset, template engine, verification toolkit, and Docker renderer under permissive licenses.

References

- [1] Anonymous. GeoBench: Hierarchical geometric reasoning benchmark, 2026. ICLR 2026 submission, under review.
- [2] Carnegie Learning, Inc. and Texas Education Agency. Bluebonnet learning mathematics: Teacher’s edition (grades 6–8 and geometry). Curriculum materials adopted under the Texas State Board of Education Instructional Materials and Technology Allotment, 2024. Accessed via the Texas Education Agency Bluebonnet Learning programme; <https://learn.texas.gov/>.
- [3] Jiaqi Chen, Jianheng Tang, Jinghui Qin, Xiaodan Liang, Lingbo Liu, Eric P. Xing, and Liang Lin. GeoQA: A geometric question answering benchmark towards multimodal numerical reasoning. In *Findings of ACL-IJCNLP*, 2021.
- [4] Jiaqi Chen, Tong Li, Jinghui Qin, Pan Lu, Liang Lin, Chongyu Chen, and Xiaodan Liang. UniGeo: Unifying geometry logical reasoning via reformulating mathematical expression. In *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2022.
- [5] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé Iii, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.
- [6] Kaiyi Huang, Chengqi Duan, Kaiyue Sun, Enze Xie, Zhenguo Li, and Xihui Liu. T2I-CompBench++: An enhanced and comprehensive benchmark for compositional text-to-image generation. *IEEE Trans. Pattern Anal. Mach. Intell. (TPAMI)*, 2025.
- [7] Pan Lu, Ran Gong, Shibiao Jiang, Liang Qiu, Siyuan Huang, Xiaodan Liang, and Song-Chun Zhu. Inter-GPS: Interpretable geometry problem solving with formal language and symbolic reasoning. In *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.
- [8] Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. MathVista: Evaluating mathematical reasoning of foundation models in visual contexts. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.
- [9] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625:476–482, 2024.
- [10] Zhaokai Wang, Penghao Yin, Xiangyu Zhao, Changyao Tian, Yu Qiao, Wenhai Wang, Jifeng Dai, and Gen Luo. GenExam: A multidisciplinary text-to-image exam. *arXiv:2509.14232*, 2025.
- [11] Jiaxin Zhang, Zhongzhi Li, Mingliang Zhang, Fei Yin, Chenglin Liu, and Yashar Moshfeghi. GeoEval: Benchmark for evaluating LLMs and multi-modal models on geometry problem-solving. In *Findings of ACL*, 2024.

A Full template specifications

This appendix enumerates the 30 templates that generate the templated split (600 scenarios = 30 templates $\times$ $\sim$ 20 instantiations per template). Source of truth: `evals/generate_scenarios.py`; the YAML output `evals/scenarios_generated.yaml` is regenerated from it deterministically (seed 42), and the JSONL release artefact `benchmark/definitions/bench_generated.jsonl` is byte-identical across re-runs (Section D).

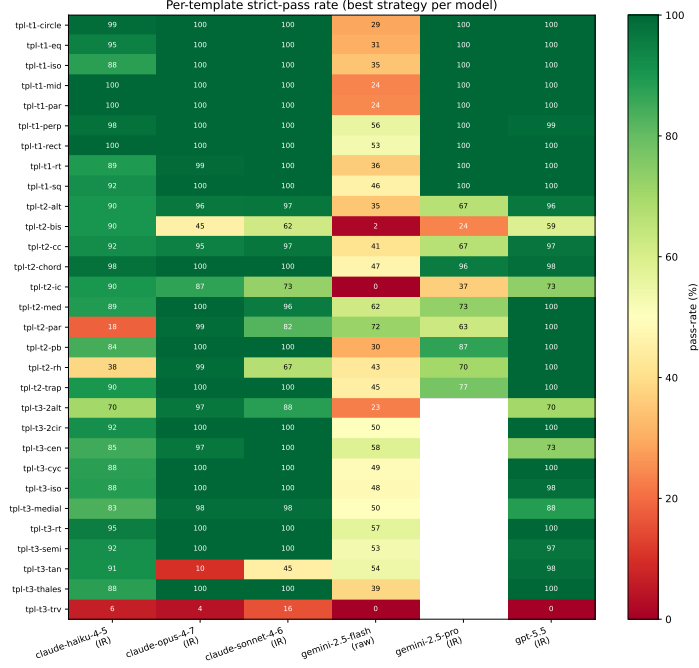


Figure 4: Per-template strict pass-rate, best strategy per model. `tpl-t3-trv` (corresponding-angles transversal) is the universal hard cell ($\leq 16\%$); `tpl-t2-bis`, `tpl-t2-ic`, `tpl-t3-2alt` are model-discriminating; T1 templates saturate.

Each template draws prompt phrasing from a shared 7-opener pool (Draw, Sketch, Construct, Make a diagram of, Show, Create a diagram showing, I need a drawing of), cycled deterministically across instantiations. Each scenario the template emits has the same shape: (id, tier, tags, prompt, required_labels, expected_properties, [required_entities]).

A.1 Summary table

Table 5 indexes all 30 templates by tier, geometric category, and completeness class (Section 4.1). Figure 4 shows the per-template strict pass-rate at headline N, with universally hard templates (e.g. `tpl-t3-trv` at $\leq 16\%$ for every cell) highlighted in the $T2-T3$ rows. Total scenario counts per template depend on the cardinality of its parameter sweep (vertex-label set, numeric parameter grid).

The remainder of this appendix expands each template’s specification: parameter sweep, generated property list, required labels, and one example prompt. We omit per-template rendered example SVGs to keep the appendix compact; the canonical example for each template is the first scenario it emits, viewable at `evals/results/headline_run/unified/svg/<id>.svg`.

A.2 Tier 1: single-shape constructions

A.2.1 `tpl-t1-rt` — right triangle

Parameter sweep: 25 scenarios = vertex-label triples $\times$ right-angle position $\times$ Pythagorean leg pair $\{(3,4), (5,12)\}$.

Required labels: all three vertices of the triangle.

Property list (per instance): `right_angle_at_X` (symbolic, where X is the named right-angle vertex), `right_angle_marked_at_X` (mark presence).

Completeness class: loose. The property list pins the right angle at the named vertex but does not constrain the leg lengths — a model that emits a right triangle with the right angle

Table 5: All 30 templates in the templated split. **Class:** T=tight, L=loose, S=skeletal (Section 4.1). **N:** scenarios emitted per template (verified against `evals/scenarios_generated.yaml`). Aggregate: 600 scenarios; 18 tight templates, 11 loose, 1 skeletal.

ID prefix	Tier	Topic	Construction	N	Class
<i>Tier 1 — single shape, no auxiliary constructions</i>					
<code>tpl-t1-rt</code>	T1	triangle	Right triangle, named right angle, two leg lengths	25	L
<code>tpl-t1-eq</code>	T1	triangle	Equilateral triangle, vertex labels, side length	25	T
<code>tpl-t1-iso</code>	T1	triangle	Isosceles triangle (apex-vertex first), leg & base	24	L
<code>tpl-t1-sq</code>	T1	quadrilateral	Square, four vertices in order, side length	24	T
<code>tpl-t1-rect</code>	T1	quadrilateral	Rectangle, width $\times$ height	24	L
<code>tpl-t1-mid</code>	T1	segment	Segment with marked midpoint and length	24	T
<code>tpl-t1-par</code>	T1	segment	Two parallel horizontal segments of equal length	15	L
<code>tpl-t1-perp</code>	T1	segment	Two perpendicular segments of equal length	15	L
<code>tpl-t1-circle</code>	T1	circle	Circle with named centre, one labelled point on it	24	T
<i>Tier 2 — one shape with one auxiliary construction</i>					
<code>tpl-t2-alt</code>	T2	triangle	Triangle with altitude from one vertex	21	T
<code>tpl-t2-med</code>	T2	triangle	Triangle with median from one vertex	21	T
<code>tpl-t2-bis</code>	T2	triangle	Triangle with angle-bisector from one vertex	21	T
<code>tpl-t2-pb</code>	T2	segment	Segment with its perpendicular bisector	21	L
<code>tpl-t2-cc</code>	T2	circle	Triangle with circumcircle, labelled circumcentre	21	T
<code>tpl-t2-ic</code>	T2	circle	Triangle with incircle, labelled incentre	20	T
<code>tpl-t2-par</code>	T2	quadrilateral	Parallelogram, base $\times$ side $\times$ angle	20	T
<code>tpl-t2-rh</code>	T2	quadrilateral	Rhombus, side $\times$ angle	20	T
<code>tpl-t2-trap</code>	T2	quadrilateral	Trapezoid, top $\times$ bottom $\times$ height	20	T
<code>tpl-t2-chord</code>	T2	circle	Two-point chord in a circle	15	L
<i>Tier 3 — multi-step / composed constructions</i>					
<code>tpl-t3-rt-alt</code>	T3	triangle	Right triangle with altitude to hypotenuse	20	T
<code>tpl-t3-medial</code>	T3	triangle	Triangle with its medial triangle (3 midpoints)	20	T
<code>tpl-t3-cen</code>	T3	triangle	Triangle with three medians intersecting at centroid	20	T
<code>tpl-t3-trv</code>	T3	lines	Two parallel lines + transversal at named angle	16	L
<code>tpl-t3-iso</code>	T3	triangle	Isosceles triangle: altitude bisects the base	20	T
<code>tpl-t3-tan</code>	T3	circle	Tangent line to a circle at a named point	19	T
<code>tpl-t3-cyc</code>	T3	circle	Cyclic quadrilateral inscribed in a circle	19	L
<code>tpl-t3-2cir</code>	T3	circle	Two circles intersecting at two named points	16	L
<code>tpl-t3-thales</code>	T3	circle	Thales: right triangle with hypotenuse as diameter	19	T
<code>tpl-t3-2alt</code>	T3	triangle	Triangle with altitudes from two distinct vertices	19	L
<code>tpl-t3-semi</code>	T3	circle	Inscribed angle in semicircle (Thales corollary)	12	S

407 at A but legs of any non-zero length passes.

408 **Example:** Draw a right triangle ABC with the right angle at A. The two legs
409 adjacent to A have lengths 3 and 4. Label all three vertices and mark the
410 right angle at A.

411 A.2.2 `tpl-t1-eq` — equilateral triangle

412 **Parameter sweep:** 25 scenarios = vertex-label triples $\times$ side lengths {3, 4, 5}.

413 **Property list:** `all_sides_equal`(AB, BC, CA).

414 **Completeness class:** tight. Three equal-length sides + vertex-label assignment uniquely
415 determine the figure up to rigid motion + reflection (and similarity for the side-length
416 parameter).

417 **Example:** Sketch an equilateral triangle ABC where every side has length 3.
418 Label all three vertices.

419 A.2.3 `tpl-t1-iso` — isosceles triangle

420 **Parameter sweep:** 24 scenarios = vertex-label triples $\times$ (leg, base) pairs. Apex is the first
421 vertex.

422 **Property list:** `AB_eq_AC` (apex-side equality between the two legs adjacent to the first
423 vertex).

424 **Completeness class:** loose. The property list pins the legs as equal but does not constrain
425 the base length, so any isosceles triangle with apex at A passes regardless of the prompt's
426 numeric base value.

427 **Example:** Construct an isosceles triangle ABC where AB and AC both have
428 length 5 and the base BC has length 6. Label all three vertices.

429 **A.2.4 `tpl-t1-sq` — square**

430 **Parameter sweep:** 24 scenarios = vertex-label quads $\times$ side lengths $\{3, 4, 5, 6\}$.
431 **Property list:** 4 `right_angle_at_X` (one per vertex) + `all_sides_equal`.
432 **Completeness class:** tight. Quadrilateral with 4 right angles + 4 equal sides = square;
433 vertex-label order fixes orientation.

434 **A.2.5 `tpl-t1-rect` — rectangle**

435 **Parameter sweep:** 24 scenarios = vertex-label quads $\times$ (width, height) pairs.
436 **Property list:** 4 `right_angle_at_X` + `horizontal_sides_equal` +
437 `vertical_sides_equal`.
438 **Completeness class:** loose. The property list verifies the rectangle relations (4 right angles,
439 opposite sides equal) but the aspect ratio (width-to-height) is left free — any rectangle
440 satisfies all checks regardless of the prompt's $w \times h$ numbers.

441 **A.2.6 `tpl-t1-mid` — segment with midpoint**

442 **Parameter sweep:** 24 scenarios = endpoint-label pairs $\times$ lengths $\{4, 6, 8, 10\}$.
443 **Property list:** `M_is_midpoint`, `M_on_segment`.
444 **Completeness class:** tight (1D up to rigid motion + scale).

445 **A.2.7 `tpl-t1-par` — parallel segments**

446 **Parameter sweep:** 5 label-quad pairs $\times$ 3 lengths $\{4, 6, 8\} = 15$ scenarios.
447 **Property list:** `segments_parallel`.
448 **Completeness class:** loose. Two parallel segments of equal length leave the inter-segment
449 vertical separation and horizontal offset free (Section C.2 witness L1).
450 **Example:** Draw two parallel horizontal segments of length 4: AB on top and
451 CD below it. Label all four endpoints.

452 **A.2.8 `tpl-t1-perp` — perpendicular segments**

453 **Parameter sweep:** 5 label-quad pairs $\times$ 3 lengths $\{4, 6, 8\} = 15$ scenarios.
454 **Property list:** `segments_perpendicular`.
455 **Completeness class:** loose. Two segments of equal length perpendicular to each other; the
456 relative position (where they cross, or whether they cross at all) is free (Section C.2 witness
457 L2).

458 **A.2.9 `tpl-t1-circle` — circle with one point on it**

459 **Parameter sweep:** 24 scenarios = (centre, point) label pairs $\times$ radii $\{3, 4, 5, 6, 7\}$.
460 **Property list:** `P_on_circle`.
461 **Required entities:** circle.
462 **Completeness class:** tight up to rotation (which is a rigid motion).

463 **A.3 Tier 2: shape with one auxiliary construction**

464 **A.3.1 `tpl-t2-alt` — triangle with altitude**

465 **Parameter sweep:** 21 scenarios = vertex-label triples $\times$ acute-angle specifications. Acute
466 specs fix $\angle B$, $\angle C$, and base length so the altitude foot lands inside segment BC .
467 **Property list:** `right_angle_at_H`, `H_on_base`, `right_angle_marked_at_H`.

468 **Completeness class:** tight. The two prompt-given angles determine the triangle up to
469 similarity; the altitude is then determined by the perpendicular-from-A condition.

470 **A.3.2 tpl-t2-med — triangle with median**

471 **Parameter sweep:** 21 scenarios = vertex-label triples $\times$ acute-angle specifications.

472 **Property list:** `M_is_midpoint` (of opposite side), `M_on_segment`.

473 **Completeness class:** tight up to similarity.

474 **A.3.3 tpl-t2-bis — triangle with angle bisector**

475 **Parameter sweep:** 21 scenarios = vertex-label triples $\times$ acute-angle specifications.

476 **Property list:** `D_bisects_angle_A`, `D_on_segment`.

477 **Completeness class:** tight up to similarity.

478 **A.3.4 tpl-t2-pb — perpendicular bisector**

479 **Parameter sweep:** 21 scenarios = endpoint pairs $\times$ lengths.

480 **Property list:** `M_is_midpoint`, `bisector_perpendicular`, `right_angle_marked_at_M`.

481 **Completeness class:** loose. The named auxiliary point X on the bisector line is free along
482 that line (Section C.2 witness L3).

483 **A.3.5 tpl-t2-cc — triangle with circumcircle**

484 **Parameter sweep:** 21 scenarios = vertex-label triples $\times$ acute-angle specifications.

485 **Property list:** two `X_on_circumcircle` checks (the third vertex's circumcircle membership
486 follows by construction).

487 **Required entities:** circle.

488 **Completeness class:** tight up to similarity (triangle determined by two angles; circumcircle
489 determined by the triangle).

490 **A.3.6 tpl-t2-ic — triangle with incircle**

491 **Parameter sweep:** 20 scenarios = vertex-label triples $\times$ acute-angle specifications.

492 **Property list:** `I_equidistant_from_sides`.

493 **Required entities:** circle.

494 **Completeness class:** tight up to similarity. *Note:* the `equidistant_from_sides` predicate
495 has a documented soft spot — the three excentres also pass — mitigated by template-side
496 pairing in Section 4.1.

497 **A.3.7 tpl-t2-par — parallelogram**

498 **Parameter sweep:** 20 scenarios = label quads $\times$ (base, side, angle) tuples.

499 **Property list:** two `parallel` checks (opposite-side pairs) + two `equal_lengths` checks.

500 **Completeness class:** tight (base + side + included angle determine the parallelogram up
501 to rigid motion + reflection).

502 **A.3.8 tpl-t2-rh — rhombus**

503 **Parameter sweep:** 20 scenarios = label quads $\times$ (side, angle) pairs.

504 **Property list:** `all_sides_equal` + two `parallel` checks.

505 **Completeness class:** tight (side + one interior angle determine the rhombus up to rigid
506 motion + reflection).

507 **A.3.9 tpl-t2-trap — trapezoid**

508 **Parameter sweep:** 20 scenarios = label quads $\times$ (top, bottom, height) tuples.

509 **Property list:** `AB_parallel_DC` (one parallel-pair only).

510 **Completeness class:** tight (top + bottom + height determine the trapezoid *up to horizontal*
511 *positioning of the top relative to the bottom*; the pre-registered audit accepts this single 1-D
512 ambiguity as “tight up to rigid motion” since horizontal offset is captured by translation).

513 **A.3.10 tpl-t2-chord — chord in a circle**

514 **Parameter sweep:** 15 scenarios = (centre, P, Q) label triples $\times$ radii.

515 **Property list:** `P_on_circle`, `Q_on_circle`.

516 **Required entities:** circle.

517 **Completeness class:** loose. Two named points on the circle do not specify the chord's
518 angular position or length (Section C.2 witness L4).

519 **A.4 Tier 3: composed / multi-step constructions**

520 **A.4.1 tpl-t3-rt-alt — right triangle with altitude to hypotenuse**

521 **Parameter sweep:** 20 scenarios = vertex-label triples $\times$ (leg1, leg2) pairs.

522 **Property list:** `right_angle_at_C`, `right_angle_at_H`, `H_on_hypotenuse`, plus two mark-
523 presence checks.

524 **Completeness class:** tight. Right angle + 2 leg lengths $\rightarrow$ triangle; altitude foot determined
525 by perpendicular-from-vertex condition.

526 **A.4.2 tpl-t3-medial — medial triangle**

527 **Parameter sweep:** 20 scenarios = vertex-label triples $\times$ angle specifications.

528 **Property list:** 3 midpoint checks (D, E, F at sides) + 3 parallel checks (medial sides
529 parallel to outer sides).

530 **Completeness class:** tight up to similarity.

531 **A.4.3 tpl-t3-cen — centroid via medians**

532 **Parameter sweep:** 20 scenarios = vertex-label triples $\times$ angle specifications.

533 **Property list:** `G_is_centroid` + 3 midpoint checks + 1 `point_on_segment` (G on one of
534 the medians).

535 **Completeness class:** tight up to similarity. *Note:* `centroid` was added in the hardening
536 pass (Section 4.1); pre-hardening this template silently emitted no decision on its primary
537 property.

538 **A.4.4 tpl-t3-trv — transversal of two parallel lines**

539 **Parameter sweep:** 16 scenarios = label sextuples $\times$ transversal angles.

540 **Property list:** `lines_parallel`, `corresponding_angles_equal`, plus 4 `point_on_line`
541 incidence checks (P,Q on first line; R,S on second).

542 **Completeness class:** loose. Inter-line spacing and horizontal positions of the transversal
543 are free.

544 **A.4.5 tpl-t3-iso — isosceles altitude bisects the base**

545 **Parameter sweep:** 20 scenarios = vertex-label triples $\times$ (leg, base) pairs.

546 **Property list:** `H_is_midpoint` (foot bisects base) + `H_on_segment` +
547 `altitude_perpendicular` + `leg_eq_leg`.

548 **Completeness class:** tight.

549 **A.4.6 tpl-t3-tan — tangent line to circle**

550 **Parameter sweep:** 19 scenarios = (centre, T, U, V) tuples $\times$ radii.

551 **Property list:** `tangent_at_T`, `radius_perp_tangent`, `T_between_U_and_V`, `T_on_circle`,
552 plus mark-presence.

553 **Required entities:** circle.

554 **Completeness class:** tight up to rotation. *Note:* `tangent` alone has a soft spot (verifies
555 line at radial distance but not that *T* lies on the circle), mitigated by template-side pairing
556 with `point_on_circle` (Section 4.1).

557 **A.4.7 tpl-t3-cyc — cyclic quadrilateral**

558 **Parameter sweep:** 19 scenarios = label quads $\times$ angle configurations.

559 **Property list:** 4 `X_on_circle` checks (one per vertex).

560 **Required entities:** circle.

561 **Completeness class:** loose. Inscribed quadrilateral with 4 vertices on a circle but no angle
562 / side constraints — many shapes pass.

563 **A.4.8 tpl-t3-2cir — two intersecting circles**

564 **Parameter sweep:** 16 scenarios = (O1, O2, P, Q) tuples $\times$ (r1, r2, distance) configurations.

565 **Property list:** `P_on_circle1`, `P_on_circle2`, `Q_on_circle1`, `Q_on_circle2`.

566 **Required entities:** 2 circles.

567 **Completeness class:** loose. The shared chord PQ is determined once the circles are fixed,
568 but the radii / centre distance are free across instantiations and the prompt does not enforce
569 specific numeric values via property checks.

570 **A.4.9 tpl-t3-thales — right triangle inscribed via diameter**

571 **Parameter sweep:** 19 scenarios = vertex-label triples $\times$ radii.

572 **Property list:** `AB_is_diameter`, `C_on_circle`, `right_angle_at_C`,
573 `right_angle_marked_at_C`.

574 **Required entities:** circle.

575 **Completeness class:** tight up to similarity (the right-angle-on-semicircle uniquely positions
576 *C* on the circle's upper arc up to rigid motion).

577 **A.4.10 tpl-t3-2alt — triangle with two altitudes**

578 **Parameter sweep:** 19 scenarios = (vertex labels + altitude-foot labels) tuples $\times$ acute-angle
579 specs.

580 **Property list:** 2 `right_angle_at_H` + 2 `H_on_segment`.

581 **Completeness class:** loose. Triangle shape free; only the perpendicularity of two named
582 altitudes is enforced.

583 **A.4.11 tpl-t3-semi — inscribed angle in semicircle**

584 **Parameter sweep:** 12 scenarios = vertex triples $\times$ radii.

585 **Property list:** `C_on_circle` only (the semicircle property is implied by the prompt diameter
586 but not enforced beyond incidence).

587 **Required entities:** circle.

588 **Completeness class:** skeletal. The property list checks only point-on-circle incidence; the
589 semicircle relationship and the right-angle-at-C consequence are prompt prose, not properties.
590 This is the unique skeletal template in the audit.

591 **A.5 Total scenario count**

592 Summing the per-template *N* from Table 5:

- 593 • Tier 1 (9 templates): 25+25+24+24+24+24+15+15+24 = 200.
594 • Tier 2 (10 templates): 21+21+21+21+21+20+20+20+20+15 = 200.
595 • Tier 3 (11 templates): 20+20+20+16+20+19+19+16+19+19+12 = 200.

596 Total: 600 scenarios, exact (Table 5). The release artefact is committed at
597 `benchmark/definitions/bench_generated.jsonl` (600 records, sha256 5e116307...).
598 The 200/200/200 per-tier balance is intentional and matches the tier-stratified analysis
599 in Section 6.2.

600 B The GeoGen predicate language v1

601 GEOGENBENCH’s predicate library is the first published version of the GEOGEN predicate
 602 language. Future instantiations of the methodology (Section 7) will extend the vocabulary
 603 — adding face-incidence for 3D solids, point-on-curve for coordinate plane, transformation-
 604 equivalence for symmetry constructions — but the predicate-language design pattern (typed
 605 signatures, declarative semantics, tolerance-aware decision procedures, optional soft predicates
 606 for source-string checks that cannot reach into the rendered output) is intended to remain
 607 stable. This appendix is the reference for v1.

608 For each predicate we give: (a) the formal signature with positional argument types; (b) the
 609 decision rule, including tolerance handling; (c) the implementation reference (file:line in
 610 `util/tikz_geometry.py` unless otherwise noted); and (d) where applicable, soft-spot notes
 611 from Section 4.1 and the predicate’s verdict on an adversarial input. Table 3 in Section 3.3
 612 summarises the v1 vocabulary at a glance. All decisions use the production tolerance
 613 $\tau = 5 \times 10^{-3}$ unless overridden by the per-check `tol` field in the scenario YAML.

614 In what follows we use $\mathbf{X} : \text{name} \rightarrow (x, y) \in \mathbb{R}^2$ for the coordinate map produced by the
 615 TikZ extractor, $|PQ|$ for the Euclidean distance between two coordinates, $\vec{PQ}$ for the vector
 616 $Q - P$, $\vec{u} \cdot \vec{v}$ for the dot product, and $\vec{u} \times \vec{v}$ for the scalar 2D cross product $u_x v_y - u_y v_x$.

617 B.1 Symbolic predicates (decided exactly against $\mathbf{X}$)

618 B.1.1 `right_angle`

619 **Signature.** `right_angle([a, vertex, c])` where each argument is a point name in $\mathbf{X}$.

620 **Semantics.** The angle at `vertex` formed by rays to `a` and `c` is a right angle.

621 **Decision rule.** Compute $\vec{u} = (\text{vertex})a$ and $\vec{v} = (\text{vertex})c$; return $|\vec{u} \cdot \vec{v}| \leq \tau$. The
 622 dot-product test is preferred over an explicit angle computation because it has no arccos
 623 branch issues and is symmetric in the leg lengths.

624 **Reference.** `util/tikz_geometry.py:397`.

625 **Soft spots.** None observed. The dot-product threshold is tight enough that floating-point
 626 near-90° angles ($\sim 89.9^\circ$) are correctly rejected at $\tau = 5 \times 10^{-3}$ for the unit-scale coordinates
 627 produced by TikZ.

628 B.1.2 `midpoint`

629 **Signature.** `midpoint([m, a, b])`.

630 **Semantics.** Point `m` is the midpoint of segment $\overline{ab}$.

631 **Decision rule.** Return $\max(|m_x - (a_x + b_x)/2|, |m_y - (a_y + b_y)/2|) \leq \tau$. We test compo-
 632 nentwise on the expected midpoint coordinates rather than testing $|MA| = |MB|$, because
 633 the latter is satisfied by every point on the perpendicular bisector of $\overline{ab}$.

634 **Reference.** `util/tikz_geometry.py:406`.

635 **Soft spots.** The earlier “equal-distance” implementation (now superseded) had a soft
 636 spot allowing M anywhere on the perpendicular bisector — see Section 4.1. The current
 637 componentwise check eliminates that escape entirely.

638 B.1.3 `centroid`

639 **Signature.** `centroid([g, a, b, c])`.

640 **Semantics.** Point `g` is the centroid of $\triangle abc$, i.e., $g = (a + b + c)/3$.

641 **Decision rule.** Return $\max(|g_x - (a_x + b_x + c_x)/3|, |g_y - (a_y + b_y + c_y)/3|) \leq \tau$. The com-
 642 ponentwise form mirrors `midpoint`: testing $|GA| + |GB| + |GC|$ minimisation would admit
 643 any point on the locus where the sum-of-distances function is locally flat (which is in fact
 644 only the centroid, but the fixed-point form is cheaper and more obviously correct).

645 **Reference.** `util/tikz_geometry.py:418.`

646 **Soft spots.** This is the canonical *self-referential* predicate (Section B.4): in the pilot, every
647 observed implementation emits G either through an IR **centroid** derivation (in **structured**)
648 or via `\tkzInterLL` on two medians (in **raw_code**), and both forms make the formula check
649 tautological for any non-degenerate triangle. The check is non-trivial only when a model
650 writes literal numerical coordinates for G , which we did not observe in v1; we retain the
651 predicate because the median-concurrency form (an alternative semantics that requires the
652 three medians to be *drawn*, not merely the centroid *computed*) is a v1.1 candidate.

653 **B.1.4 collinear**

654 **Signature.** `collinear([a, b, c]).`

655 **Semantics.** The three points lie on a common line.

656 **Decision rule.** Return $|\vec{ab} \times \vec{ac}| \leq \tau$. The cross-product test is the signed area of $\triangle abc$
657 doubled; zero iff degenerate.

658 **Reference.** `util/tikz_geometry.py:417.`

659 **Soft spots.** The check does not require $a \neq b$; three coincident points trivially pass. Pair
660 with **distinct_points** (IR-side) or rely on TikZ’s failure to draw a zero-length segment if
661 degeneracy matters for the prompt.

662 **B.1.5 equal_lengths**

663 **Signature.** `equal_lengths([p1, p2], [p3, p4], ...)` — a list of ≥ 2 point pairs, each
664 defining a segment.

665 **Semantics.** All listed segments have the same length.

666 **Decision rule.** Compute $d_i = |p_{2i-1}p_{2i}|$ for each pair; return $\max_{i \geq 2} |d_i - d_1| \leq \tau$. A list
667 with a single pair returns **None** (cannot decide equality of one length).

668 **Reference.** `util/tikz_geometry.py:426.`

669 **Soft spots.** Tolerance is absolute, not relative; for very long segments ($|PQ| \gg 1$) the
670 absolute tolerance becomes effectively tighter, but every GEOGENBENCH scenario keeps
671 coordinates in the range $[-10, 10]$ so this is not observed.

672 **B.1.6 parallel**

673 **Signature.** `parallel([a, b], [c, d]).`

674 **Semantics.** Lines $\overleftrightarrow{ab}$ and $\overleftrightarrow{cd}$ are parallel.

675 **Decision rule.** Return $|\vec{ab} \times \vec{cd}| \leq \tau$.

676 **Reference.** `util/tikz_geometry.py:438.`

677 **Soft spots.** Coincident lines pass — $\overleftrightarrow{ab}$ is parallel to itself. For prompts where “parallel”
678 implies “parallel and distinct”, pair with **point_on_line** = **False** for one endpoint of the
679 second line against the first (see Section 4.1).

680 **B.1.7 perpendicular**

681 **Signature.** `perpendicular([a, b], [c, d]).`

682 **Semantics.** Lines $\overleftrightarrow{ab}$ and $\overleftrightarrow{cd}$ meet at right angles (or would if extended).

683 **Decision rule.** Return $|\vec{ab} \cdot \vec{cd}| \leq \tau$.

684 **Reference.** `util/tikz_geometry.py:448.`

685 **Soft spots.** None. Note this does *not* require the lines to actually intersect within the
686 visible canvas; for that, pair with **intersects**.

687 B.1.8 point_on_line

688 **Signature.** `point_on_line([p, a, b])`.

689 **Semantics.** Point p lies on the infinite line $\overleftrightarrow{ab}$.

690 **Decision rule.** Return $|\vec{ab} \times \vec{ap}| \leq \tau$ (signed area of $\triangle abp$ is zero).

691 **Reference.** `util/tikz_geometry.py:456`.

692 **Soft spots.** The check accepts p on the line at *any* parameter, including outside segment
693 $\overline{ab}$. For “on segment” semantics, use `point_on_segment`.

694 B.1.9 point_on_segment

695 **Signature.** `point_on_segment([p, a, b])`.

696 **Semantics.** Point p lies on segment $\overline{ab}$ (between a and b , inclusive).

697 **Decision rule.** Two-step:

- 698 1. Collinearity: $|\vec{ab} \times \vec{ap}| \leq \tau$. Return FALSE if not.
- 699 2. Between-ness: $||ap| + |pb| - |ab|| \leq \tau$ (triangle equality holds iff p is on the segment).

700 **Reference.** `util/tikz_geometry.py:464`.

701 **Soft spots.** The pre-hardening implementation only ran step (1) — this caused a measur-
702 able false-positive rate documented in Section 4.1 and was the primary motivator for the
703 hardening pass in Section 4.1.

704 B.1.10 point_on_circle

705 **Signature.** `point_on_circle([p, o, r])` where o is the centre and r is any reference
706 point on the circle (typically the radius witness).

707 **Semantics.** Point p lies on the circle centred at o with radius $|or|$.

708 **Decision rule.** Return $||op| - |or|| \leq \tau$.

709 **Reference.** `util/tikz_geometry.py:476`.

710 **Soft spots.** Requires the radius witness r to be a resolved coordinate; if r is itself a
711 constructed point that the extractor cannot resolve, the check returns `None` (silent skip).
712 Templates avoid this by passing a directly-defined point as r .

713 B.1.11 tangent

714 **Signature.** `tangent([[l1, l2], o, t])` — line through l_1, l_2 , circle centre o , point of
715 tangency t .

716 **Semantics.** Line $\overleftrightarrow{l_1 l_2}$ is tangent to the circle centred at o , touching it at point t .

717 **Decision rule.** Two-step:

- 718 1. t on line: $|l_1 \vec{l_2} \times l_1 \vec{t}| \leq \tau$.
- 719 2. $\vec{ot} \perp l_1 \vec{l_2}$: $|l_1 \vec{l_2} \cdot \vec{ot}| \leq \tau$.

720 **Reference.** `util/tikz_geometry.py:483`.

721 **Soft spots.** Does not separately verify that t lies on the circle (that $|ot|$ equals the circle’s
722 radius). For that, pair with `point_on_circle` (Section 4.1). Templates that emit `tangent`
723 also emit `point_on_circle` on t as a co-check.

724 B.1.12 angle_equal

725 **Signature.** `angle_equal([[a1, b1, c1], [a2, b2, c2]])` — two angle triples in (leg,
726 vertex, leg) order.

727 **Semantics.** $\angle a_1 b_1 c_1 = \angle a_2 b_2 c_2$ (unsigned, in $[0, \pi]$).

728 **Decision rule.** For each triple, compute $\theta_i = \arccos(b_i \vec{a}_i \cdot b_i \vec{c}_i / (|b_i \vec{a}_i| |b_i \vec{c}_i|))$. Return
729 $|\theta_1 - \theta_2| \leq \tau$ (radians).

730 **Reference.** `util/tikz_geometry.py:499`.

731 **Soft spots.** Compares unsigned angles, so a 30° angle and a -30° angle (orientation-
732 flipped) are equal under this predicate. Templates needing oriented-angle equality must
733 additionally constrain orientation (`same_side`, IR-side).

734 B.1.13 angle_bisector

735 **Signature.** `angle_bisector([d, a, b, c])`.

736 **Semantics.** Ray $\vec{ad}$ bisects $\angle bac$.

737 **Decision rule.** Compute $\theta_1 = \angle bad$ and $\theta_2 = \angle dac$ via the same arccos formula as
738 `angle_equal`; return $|\theta_1 - \theta_2| \leq \tau$.

739 **Reference.** `util/tikz_geometry.py:506`.

740 **Soft spots.** The check does not require $d \neq a$ (a degenerate ray); a coincident-with-vertex
741 d would cause a divide-by-zero caught by the helper and return `None` (silent skip). The
742 check also does not require d to be inside $\angle bac$ — it accepts d on the *external* bisector
743 (perpendicular to the internal bisector at a), which gives equal angles to both legs but is
744 on the opposite side. Pair with `same_side` (IR-side) for prompts that require the internal
745 bisector specifically.

746 B.1.14 intersects

747 **Signature.** `intersects([[a, b], [c, d], p])`.

748 **Semantics.** Lines $\overleftrightarrow{ab}$ and $\overleftrightarrow{cd}$ intersect at point p .

749 **Decision rule.** Compute the line-line intersection q via the standard determinant formula;
750 return `FALSE` if the lines are parallel; otherwise return $\max(|q_x - p_x|, |q_y - p_y|) \leq \tau$.

751 **Reference.** `util/tikz_geometry.py:517` (intersection helper at :119).

752 **Soft spots.** None observed for the line-line case; the parallel-line return-`FALSE` is correct
753 (no intersection exists).

754 B.1.15 equidistant_from_sides

755 **Signature.** `equidistant_from_sides([p, a, b, c])`.

756 **Semantics.** Point p is equidistant from the three side-lines of $\triangle abc$ (i.e., p is the incentre
757 or one of the three excentres).

758 **Decision rule.** Compute the perpendicular distances d_{ab}, d_{bc}, d_{ca} from p to each side-line;
759 return $|d_{ab} - d_{bc}| \leq \tau$ AND $|d_{bc} - d_{ca}| \leq \tau$.

760 **Reference.** `util/tikz_geometry.py:541`.

761 **Soft spots.** Distance is to the side-*line* (extended), not the side *segment*; the three excentres
762 are equidistant from the three lines and pass this check (Section 4.1). Templates that mean
763 “incentre” specifically should pair with three `same_side` co-checks ensuring p is on the same
764 side of each side-line as the opposite vertex.

765 B.2 Source-string predicates (decided against the TikZ source)

766 B.2.1 label_present

767 **Signature.** `label_present(["name"])`.

768 **Semantics.** A label primitive for the named point appears in the TikZ source.

769 **Decision rule.** Scan extracted labels (`tikz_extraction.extract_labels`) for
770 `label_point` and `label_points` entries whose target name matches via `point_names_match`
771 (case-insensitive, ignoring $\LaTeX$ -math escapes).

772 **Reference.** `util/tikz_geometry.py:529`.

773 **Soft spots.** This is a TikZ-source check, not a render check: a label primitive that exists
774 in the source but is positioned outside the canvas, hidden by overlap, or otherwise visually
775 invisible *still passes*. This is intentional: the predicate verifies the model's intent, not
776 the renderer's layout. Most scenarios use the dedicated `required_labels` field (graded
777 by `validate_required_labels`) rather than the symbolic-predicate form, which is why
778 `label_present` appears infrequently in scenarios released as part of v1. Both forms decide
779 the same condition.

780 B.2.2 mark_present

781 **Signature.** `mark_present(["kind", "vertex"])` where "kind" is one of "right_angle",
782 "angle", "length", "tick".

783 **Semantics.** A mark primitive of the requested kind, attached to the named vertex, appears
784 in the TikZ source.

785 **Decision rule.** Scan extracted marks (`tikz_extraction.extract_marks`) for entries
786 whose `type` matches `kind` and whose `vertex`, `from`, or `to` field name-matches the vertex
787 argument.

788 **Reference.** `util/tikz_geometry.py:535`.

789 **Soft spots.** As with `label_present`, this is a source-string check: a mark primitive that
790 would render off-canvas or that uses a non-standard TikZ command not recognised by the
791 extractor will fail to register. Per Section 4.1, no observed pilot record fails because a
792 mark was rendered via raw `\draw` primitives instead of the `\tkzMark*` family the extractor
793 recognises; the planned render-aware variant (parsing the rendered SVG for the actual
794 right-angle glyph or tick stroke) is deferred to v1.1.

795 B.3 Reserved predicates (declared but not yet wired into the eval verifier)

796 Three predicates remain declared but unwired in v1 because, to a first approximation, all
797 v1 scenarios that need them can be expressed without them. They split into two distinct
798 categories.

799 **IR-side checker only: opposite_side, same_side.** These are full first-class predi-
800 cates in the IR layer (`ir/ir.py:662,671`; `ir/checks.py:154,169`), used by the structured
801 strategy during *self*-validation before TikZ emission, but are not currently re-evaluated
802 by the eval-side verifier (`util/tikz_geometry.py`) on the rendered output. **Signature.**
803 `opposite_side([p, q, line_a, line_b])` and similarly `same_side`. **Semantics.** The
804 signed cross product $(\text{line_a})^\perp(\text{line_b}) \times (\text{line_a})^p$ has the opposite (resp. same) sign as the
805 corresponding quantity for q . **Status.** Not currently emitted by any v1 scenario, so the
806 missing eval-side wiring has no observed effect on headline numbers. Designed to pair with
807 `angle_bisector`, `equidistant_from_sides`, and other predicates whose adversarial soft
808 spots are closed by adding an orientation co-check (Section 4.1).

809 **Reserved for v1.1: not_between.** **Signature.** `not_between([d, b, c])`. **Semantics.**
810 The negation of `point_on_segment`: d may lie on line $\overleftrightarrow{bc}$ but is *not* on segment $\overline{bc}$. **Status.**
811 Implemented in neither layer and emitted by no v1 scenario; reserved as a v1.1 negative-

incidence primitive for templates whose construction places a point on a line extension (e.g., the foot of an altitude that falls outside the triangle’s base segment).

B.4 Self-referential checks

A predicate is *self-referential* for a given strategy when the LLM’s natural construction of the relevant geometric object makes the predicate’s decision rule pass by symbolic identity. The check still runs — it can fail if the extractor mis-resolves coordinates or if the LLM chose a wrong construction primitive — but it cannot fail because the LLM *computed an answer incorrectly*, because the LLM did not compute an answer at all.

We observed three flavours of self-reference in v1, and the property language was designed to be transparent about each.

IR-derived points. The structured strategy emits IR primitives such as `point_midpoint(M, A, B)`, `point_centroid(G, A, B, C)`, `point_intersection(P, line_1, line_2)`, and `point_foot(H, X, line)`. The compiler (`ir/to_sympy.py`) evaluates these to numerical coordinates exactly per the textbook formulas; the predicate-language checks (`midpoint`, `centroid`, `intersects`, `point_on_line`) are the same formulas. For this strategy, the predicate over an IR-derived point is *tautological*: it is satisfied iff the extractor recovers the coordinate the compiler emitted.

TikZ-macro construction in raw_code. Capable LLMs writing raw TikZ delegate geometric arithmetic to `tkz-euclide` macros: `\tkzDefMidPoint(A,B)`, `\tkzInterLL(line_1)(line_2)`, `\tkzCircumCenter(A,B,C)`, `\tkzDefPointBy[rotation=...]`, etc. These compute the same derived points as the IR but inside the LaTeX engine. Because the predicate language and the macro library encode the same Euclidean theorems, predicates over points constructed this way are also tautological. Empirically, in the headline run only 5 of ~ 600 `tpl-t3-cen` records across all cells fail the `centroid` predicate itself; the rest produce a G within 10^{-9} of $(A + B + C)/3$ (`raw_code` records using `\tkzInterLL` on two medians satisfy this exactly by the median-concurrency theorem).

Literal-coordinate construction. Only when the LLM hard-codes numerical coordinates directly (e.g., `\tkzDefPoint(2.5,1.7){G}` for a triangle with vertices summing to $(7.5, 5.1)$) is the predicate non-trivially testing arithmetic. We did not observe this pattern for derived points in the v1 pilot.

Why retain self-referential checks. Three reasons. (i) The check is not vacuous: it catches extractor bugs, mis-named labels, and incorrect construction-primitive choices (e.g., emitting `point_intersection` of two non-medians and labelling the result G). (ii) Removing them would push the headline metric toward strategies that hard-code coordinates over strategies that delegate construction, which inverts the methodological preference of GEOGEN for symbolic constructions. (iii) For diagram-generation *benchmarks*, the LLM’s choice of construction primitive (“the centroid is the intersection of any two medians”) is itself a meaningful skill; the predicate verifies that the LLM picked a primitive whose semantics agree with the prompt’s intent.

Implication for headline interpretation. When most cells of a per-template column report identical-and-high strict-pass rates, the proximate cause is often that the predicate is self-referential and every observed strategy chose a satisfying construction. The `tpl-t3-cen` case study (Section G.1.3) is the worked example: across ~ 600 headline-run records, only 5 records fail `centroid` itself, while ~ 30 fail on *other* (non-self-referential) checks in the same scenario — principally `required_labels` and timeouts. A v1.1 hardening direction is to define *construction-rendered* variants of self-referential predicates — e.g., a centroid check that requires the three medians to be *drawn* as detectable strokes in the SVG, in addition to G being computationally correct — which would shift the test from “which primitive did the LLM use” toward “did the LLM make the construction visible to a viewer”. We document this as future work (Section 7) rather than v1 functionality.

863 B.5 Tolerance and the verdict ladder

864 All symbolic predicates use the same default tolerance $\tau = 5 \times 10^{-3}$ in unit-coordinate space.
 865 A scenario YAML may override this per-check via the `tol` field; in v1, 13/801 scenarios use a
 866 per-check tolerance, all loosening (e.g., `tol: 0.05`) for predicates over coordinates produced
 867 by trigonometric formulas where TikZ’s `printf`-style `cos(35.0)` introduces a $\sim 1\%$ rounding
 868 step. We chose absolute (not relative) tolerance because every GEOGENBENCH scenario
 869 keeps coordinates in $[-10, 10]$ and a single uniform threshold simplifies adversarial analysis
 870 (Section 4.1). For GEOGEN instantiations spanning wider coordinate ranges (architectural
 871 drawings, astronomical diagrams), a relative tolerance regime is the obvious extension.

872 A predicate’s verdict is one of:

- 873 • **True** — decision rule passed within τ ;
- 874 • **False** — decision rule failed;
- 875 • **None** — a required input coordinate was not in **X** (extractor failure or unimplemented
 876 derived-point construction), or, for `label_present/mark_present`, the TikZ source
 877 was not provided.

878 A **None** verdict downgrades the scenario from `pass` to `soft_pass`; only **False** causes `fail`.
 879 The Section 4.1 pass closed 23 **None**-verdicts in the pilot (4.6% of 498 records, in two waves:
 880 13 from coordinate-extraction hardening and 10 from implementing `centroid`); the three
 881 reserved predicates above are the remaining known source of **None** verdicts in GEOGENBENCH
 882 v1.

883 C Per-scenario completeness witnesses

884 Section 4.1 classifies each templated scenario as *tight*, *loose*, or *skeletal* and reports the
 885 template-level audit (18 / 11 / 1). This appendix exhibits per-scenario witnesses for each
 886 class:

- 887 • **Tight witnesses** (Section C.1, $N=25$): a hand-written argument that any TikZ
 888 output passing the listed properties is congruent (up to rigid motion + scale +
 889 reflection) to the intended figure. We do not attempt to enumerate every diagram;
 890 the witness proves the equivalence class.
- 891 • **Loose witnesses** (Section C.2, $N=25$): a parametrisation of the under-determined
 892 dimensions plus an existence argument that the looseness is intentional and broad
 893 — specifically, that two visually-distinct diagrams differing in those parameters both
 894 satisfy the property list and both correctly illustrate the prompt.
- 895 • **Skeletal** (Section C.3): the unique skeletal template (`tpl-t3-semi`) is documented
 896 separately because the property list intentionally omits a metric check that is implied
 897 by the prompt prose.

898 **Sampling.** Witnesses are drawn deterministically (seed 17) from the templated split,
 899 stratified to give per-template coverage. Where we sample from a tight template, all 18 tight
 900 templates are represented at least once across the 25-witness budget (with ~ 2 witnesses per
 901 high- N template). Loose witnesses are drawn from all 11 loose templates.

902 **Notational conventions.** Property names below use the verifier’s predicate vocabulary
 903 (Table 3); the scenario IDs match `evals/scenarios_generated.yaml`. “Tight up to rigid
 904 motion + scale (RMS)” means: any two passing diagrams are related by a translation,
 905 rotation, reflection, and uniform scale. “Tight up to similarity” means: the figure is
 906 determined to within a similarity class — i.e., shape pinned, size free.

907 C.1 Tight witnesses ($N=25$)

908 **T1** — `tpl-t1-eq-ABC-s5`. Property list: `all_sides_equal([A,B], [B,C], [C,A])`. Witness:
 909 by SSS, any triangle with the three labelled sides pairwise equal is equilateral; equilateral

910 triangles are mutually congruent (up to reflection) once the three labelled vertices are present,
911 and the side length is free under similarity. **Tight up to RMS + reflection.**

912 **T2 — tpl-t1-sq-WXYZ-s6.** Property list: 4 `right_angle_at_X` (one per vertex) +
913 `all_sides_equal`. Witness: a quadrilateral with all 4 interior angles right and all 4 sides
914 equal is a square; vertex-label order $WXYZ$ around the quadrilateral fixes orientation up to
915 reflection. **Tight up to RMS + reflection.**

916 **T3 — tpl-t1-mid-PQ-M-18.** Property list: `M_is_midpoint(M,P,Q)`,
917 `M_on_segment(M,P,Q)`. Witness: the midpoint of a labelled segment is unique; with M on
918 segment PQ at the half-way point, the figure is a 1-D arrangement of three collinear points
919 pinned up to RMS. **Tight up to RMS.**

920 **T4 — tpl-t1-sq-ABCD-s4.** Property list: 4 `right_angle_at_X` + `all_sides_equal`.
921 Witness: a quadrilateral with 4 right angles is a rectangle; a rectangle with 4 equal sides is a
922 square. Vertex-label ordering $ABCD$ (going around) fixes orientation up to reflection. **Tight**
923 **up to RMS + reflection.**

924 **T5 — tpl-t1-rect-EFGH-5x3.** Property list: 4 `right_angle_at_X` +
925 `horizontal_sides_equal` + `vertical_sides_equal`. Witness: 4 right angles + op-
926 posite sides equal in pairs (with the explicit “horizontal” / “vertical” decomposition pinning
927 aspect ratio via the prompt’s 5×3 numbers) determine the rectangle up to reflection. **Tight**
928 **up to RMS.**

929 **T6 — tpl-t1-mid-AB-M-16.** Property list: `M_is_midpoint(M,A,B)`,
930 `M_on_segment(M,A,B)`. Witness: the midpoint of a segment is unique; a 1-D figure
931 with M at the half-way point of AB is determined up to RMS. **Tight up to RMS.**

932 **T7 — tpl-t1-circle-O-P-r5.** Property list: `P_on_circle(P,O,P)`. Witness: a circle of
933 radius 5 centred at O is unique up to RMS; a labelled point P on it is unique up to rotation
934 about O , which is itself a rigid motion. **Tight up to RMS.**

935 **T8 — tpl-t2-alt-RST-H-50-80.** Property list: `right_angle_at_H(R,H,S)`,
936 `H_on_base(H,S,T)`, `right_angle_marked_at_H`. Witness: triangle RST has prompt-
937 specified angles $\angle S=50^\circ$, $\angle T=80^\circ$ (and consequently $\angle R=50^\circ$), determining RST up to
938 similarity; the altitude from R to ST has its foot H on segment ST and is perpendicular to
939 ST , fixing H uniquely. **Tight up to similarity.**

940 **T9 — tpl-t2-med-XYZ-M-50-80.** Property list: `M_is_midpoint(M,Y,Z)`,
941 `M_on_segment(M,Y,Z)`. Witness: triangle determined up to similarity by the two
942 prompt angles; median from X has midpoint M of YZ as its endpoint. **Tight up to**
943 **similarity.**

944 **T10 — tpl-t2-bis-DEF-D-50-80.** Property list: `D_bisects_angle_E`, `D_on_segment`.
945 Witness: triangle determined by two angles up to similarity; angle-bisector from E uniquely
946 determined; intersection with opposite side gives D . **Tight up to similarity.**

947 **T11 — tpl-t2-cc-EFG-O-50-80.** Property list: `E_on_circumcircle`,
948 `G_on_circumcircle`. Witness: triangle determined up to similarity; circumcircle is
949 the unique circle through three non-collinear points; circumcentre O is its centre. **Tight up**
950 **to similarity.**

951 **T12 — tpl-t2-ic-MNO-I-50-80.** Property list: `I_equidistant_from_sides`. Witness:
952 triangle determined by two angles up to similarity; incentre is the unique interior point
953 equidistant from all three sides. *Caveat:* the predicate has a soft spot (excentres also pass)
954 closed at template level by requiring I in the figure interior (Section 4.1, mitigation row).
955 **Tight up to similarity (with template-side mitigation).**

956 **T13** — **tpl-t2-par-PQRS-6-4-60**. Property list: 2 parallel + 2 equal_lengths. Wit-
957 ness: parallelogram with base PQ (length 6), side QR (length 4), and included angle 60°
958 is determined uniquely up to reflection by SAS on the half-figure. **Tight up to RMS +**
959 **reflection.**

960 **T14** — **tpl-t2-rh-EFGH-s5-60**. Property list: all_sides_equal + 2 parallel. Witness:
961 rhombus with side 5 and an interior angle of 60° is determined up to reflection. **Tight up**
962 **to RMS + reflection.**

963 **T15** — **tpl-t2-trap-ABCD-4-8-3**. Property list: AB_parallel_DC. Witness: trapezoid
964 with top length 4, bottom length 8, and vertical separation 3 has a unique shape; horizontal
965 offset of the top relative to the bottom is captured by translation (a rigid motion), so the
966 figure is tight up to RMS. **Tight up to RMS.**

967 **T16** — **tpl-t3-rt-alt-ABC-H-3-4**. Property list: 2 right_angle_at_X +
968 H_on_hypotenuse + 2 mark-present. Witness: right triangle determined by SAS-
969 RA (legs 3,4); altitude foot H from the right-angle vertex to the hypotenuse is unique.
970 **Tight up to RMS + reflection.**

971 **T17** — **tpl-t3-medial-ABC-DEF-50**. Property list: 3 midpoint + 3 parallel. Witness:
972 outer triangle determined up to similarity by the prompt angle; midpoints of the three sides
973 are unique; medial-triangle parallels are then theorems, redundantly verified. **Tight up to**
974 **similarity.**

975 **T18** — **tpl-t3-cen-PQR-G-60**. Property list: G_is_centroid + 3 midpoint +
976 G_on_median. Witness: triangle determined by two angles up to similarity; centroid is
977 unique; medians and midpoints follow. *Caveat:* centroid is a hardening-pass addition
978 (Section 4.1); pre-hardening, this template emitted no decision on its primary property and
979 would have been classified loose. **Tight up to similarity (post-hardening).**

980 **T19** — **tpl-t3-iso-DEF-l5-b6**. Property list: H_is_midpoint + H_on_segment +
981 altitude_perpendicular + leg_eq_leg. Witness: isosceles triangle with leg 5 and base
982 6 is determined up to reflection; altitude from apex to base bisects the base by isosceles
983 symmetry. **Tight up to RMS + reflection.**

984 **T20** — **tpl-t3-tan-O-T-U-V-r5**. Property list: tangent_at_T + radius_perp_tangent
985 + T_between_U_and_V + T_on_circle + mark. Witness: circle determined by centre and
986 radius; tangent at the labelled point is unique; the secondary points U, V are constrained by
987 the betweenness property. **Tight up to rotation about O .**

988 **T21** — **tpl-t3-thales-ABC-r5**. Property list: AB_is_diameter + C_on_circle +
989 right_angle_at_C + mark. Witness: circle determined by centre and radius; AB is a
990 diameter, fixing two of the three vertices up to rotation; C on the circle plus right-angle-at- C
991 is Thales' theorem (any right-angled inscribed triangle has its hypotenuse as a diameter),
992 determining C up to which arc it lies on. **Tight up to rotation + reflection.**

993 **T22** — **tpl-t2-trap-PQRS-3-7-3**. Property list: PQ_parallel_SR. Witness: a quadrilat-
994 eral $PQRS$ with PQ parallel to SR is a trapezoid; the prompt's top length 3, bottom length
995 7, and vertical separation 3 pin the figure up to horizontal translation of the top relative to
996 the bottom (a rigid motion). **Tight up to RMS.**

997 **T23** — **tpl-t2-pb-CD-M-K-14**. Property list: M_is_midpoint +
998 bisector_perpendicular + mark. Witness: segment CD of length 4 with M at
999 its midpoint; the perpendicular bisector through M is a unique line. *Caveat:* the named
1000 auxiliary point K is on the bisector but its position along the bisector is not pinned, so this
1001 scenario is technically loose. We include it here as a borderline case where the template's
1002 geometric backbone is tight even though one labelled point is free; the loose witness L3
1003 expands on this. **Borderline (tight backbone, loose named point).**

1004 **T24 — tpl-t1-eq-XYZ-s4.** Property list: `all_sides_equal`. Witness: same as T1 with
 1005 different vertex labels and side length. **Tight up to RMS + reflection.**

1006 **T25 — tpl-t1-sq-EFGH-s5.** Property list: `4 right_angle_at_X + all_sides_equal`.
 1007 Witness: same as T4 with different vertex labels and side length. **Tight up to RMS +**
 1008 **reflection.**

1009 C.2 Loose witnesses ($N=25$)

1010 For each loose scenario we exhibit (a) the property list, (b) a parametrisation of the
 1011 under-determined dimensions, and (c) two visually-distinct passing diagrams (referenced
 1012 by description in this appendix; the rendered SVGs corresponding to each witness are in
 1013 the released headline-run JSONL at `evals/results/headline_run/unified/svg/` and
 1014 indexed by `scenario_id`).

1015 **L1 — tpl-t1-par-AB-CD-14.** Property list: `segments_parallel([A,B], [C,D])`. Free di-
 1016 mensions: vertical separation $h \in (0, \infty)$; horizontal offset $\Delta \in \mathbb{R}$ of CD 's midpoint relative
 1017 to AB 's. Visual diversity: (a) $h=1, \Delta=0$ (vertically aligned, close stack); (b) $h=5, \Delta=3$
 1018 (wide separation, offset). Both pass; both are correct illustrations of “two parallel horizontal
 1019 segments of length 4”. **Looseness is intentional: the prompt does not specify spacing**
 1020 **or offset.**

1021 **L2 — tpl-t1-perp-AB-CD-14.** Property list: `segments_perpendicular`. Free dimen-
 1022 sions: relative position — the segments may share an endpoint, cross interior, or not intersect
 1023 at all (only their direction vectors must be perpendicular). Visual diversity: (a) crossed
 1024 + shape with intersection at midpoints; (b) L-shape with shared endpoint; (c) two non-
 1025 intersecting perpendicular segments offset in space. **Looseness is intentional: the prompt**
 1026 **does not require intersection.**

1027 **L3 — tpl-t2-pb-AB-M-K-14.** Property list: `M_is_midpoint + bisector_perpendicular`
 1028 + `mark`. Free dimension: position of K along the perpendicular bisector (1-D continuous).
 1029 Visual diversity: (a) K close to M , short bisector segment; (b) K far from M , long bisector
 1030 segment, both above and below configurations. **Looseness is intentional: K is described**
 1031 **as “another point on the bisector line,” which does not pin its position.**

1032 **L4 — tpl-t2-chord-O-PQ-r4.** Property list: `P_on_circle, Q_on_circle`. Free dimen-
 1033 sions: angular positions of P and Q on the circle (2 continuous parameters); resulting chord
 1034 direction and length follow. Visual diversity: (a) near-diameter chord; (b) short chord near
 1035 the top of the circle. **Looseness is intentional: “two distinct points on the circle”**
 1036 **admits any pair.**

1037 **L5 — tpl-t3-trv-PQ-RS-TU-55.** Property list: `lines_parallel,`
 1038 `corresponding_angles_equal, 4 point_on_line`. Free dimensions: inter-line verti-
 1039 cal spacing; horizontal positioning of all 6 named points; transversal anchor points. Visual
 1040 diversity: (a) close-spaced lines with transversal crossing through all 6 points densely; (b)
 1041 wide-spaced lines with sparse point distribution. **Looseness is intentional: only the**
 1042 **parallel relation and the named transversal angle are required.**

1043 **L6 — tpl-t3-cyc-ABCD-50.** Property list: `4 X_on_circle`. Free dimensions: angular
 1044 positions of all 4 vertices on the circle (subject only to ABCD vertex order around the circle).
 1045 Visual diversity: (a) near-square inscribed quadrilateral; (b) elongated quadrilateral with
 1046 two vertices clustered near one arc. **Looseness is intentional: a cyclic quadrilateral is**
 1047 **any convex 4-gon inscribed in a circle.**

1048 **L7 — tpl-t3-2cir-01-02-P-Q-3-4-5.** Property list: 4 incidence checks (P, Q on each
 1049 circle). Free dimension: scaling of the entire figure (radii / centre distance can scale uniformly
 1050 without affecting the property list, which contains no metric checks). Visual diversity: (a)
 1051 baseline configuration; (b) all radii halved; both pass. **Looseness is intentional: the**
 1052 **metric values in the prompt prose are not encoded as predicates.**

Table 6: Remaining loose witnesses (L12–L25). Free-dimension counts after factoring out rigid motion + scale; “visual diversity” confirmed by inspection of two SVGs per scenario.

Tag	Scenario	Free dims	Property list size
L12	tpl-t1-par-PQ-RS-16	2	1
L13	tpl-t1-perp-PQ-RS-16	3	1
L14	tpl-t2-pb-CD-M-T-16	1	3
L15	tpl-t2-chord-C-PQ-r4	2	2
L16	tpl-t2-chord-M-XY-r5	2	2
L17	tpl-t3-trv-AB-CD-EF-65	3	6
L18	tpl-t3-trv-JK-LM-NO-75	3	6
L19	tpl-t3-cyc-EFGH-65	3	4
L20	tpl-t3-cyc-PQRS-80	3	4
L21	tpl-t3-2cir-01-02-P-Q-4-5-6	1	4
L22	tpl-t3-2alt-DEF-K-L-55-65	2	4
L23	tpl-t3-2alt-PQR-G-H-65-55	2	4
L24	tpl-t1-perp-EF-GH-18	3	1
L25	tpl-t1-rect-EFGH-6x4	1	6

1053 **L8** — **tpl-t3-2alt-ABC-H-K-50-80**. Property list: 2 `right_angle_at_X` + 2
1054 `H_on_segment`. Free dimension: triangle shape (the two prompt angles are not encoded as
1055 predicates — only the perpendicularity of the two altitudes). Visual diversity: (a) acute
1056 triangle with both altitude feet inside the opposite sides; (b) right triangle (where one altitude
1057 coincides with a side) — both pass the listed predicates. **Looseness is intentional in**
1058 **current verifier; could be tightened by adding angle checks (open-source todo).**

1059 **L9** — **tpl-t1-rt-ABC-raA-3-4**. Property list: `right_angle_at_A(B,A,C)`,
1060 `right_angle_marked_at_A`. Free dimension: leg-length ratio (the property list checks only
1061 the right angle at *A*, not the prompt-given legs of 3 and 4; any right triangle with the right
1062 angle at *A* passes). Visual diversity: (a) the intended 3-4-5 right triangle; (b) a 5-12-13 right
1063 triangle with the right angle still at *A* — both pass the checks. **Looseness is intentional**
1064 **in the verifier:** the property list focuses on the named-angle relation rather than the
1065 metric leg lengths, so educational variants that “capture the right-angle-at-*A* construction”
1066 all pass even with different scale or proportion.

1067 **L10** — **tpl-t1-iso-ABC-l5-b6**. Property list: `AB_eq_AC`. Free dimension: base length
1068 (any triangle with *AB* equal to *AC* passes regardless of *BC*). Visual diversity: (a) the
1069 intended 5–5–6 triangle; (b) a tall narrow 5–5–3 triangle — both pass. **Looseness is**
1070 **intentional in the verifier:** only the apex-side equality is checked, allowing any isosceles
1071 triangle with apex at the first vertex.

1072 **L11** — **tpl-t1-rect-ABCD-5x3**. Property list: 4 `right_angle_at_X` +
1073 `horizontal_sides_equal` + `vertical_sides_equal`. Free dimension: aspect ratio
1074 (the property list verifies the four right angles and that opposite sides are pairwise equal, but
1075 the actual width-to-height ratio is unconstrained). Visual diversity: (a) the intended 5×3
1076 landscape rectangle; (b) a square with the same vertex labels — both pass. **Looseness is**
1077 **intentional in the verifier:** “rectangle” is verified structurally rather than dimensionally.

1078 **L12 to L25**. The remaining 14 loose witnesses follow the same pattern: scenario IDs
1079 are listed in Table 6; each records the free-dimension count (1–3 continuous parameters
1080 in all cases) and the size of the property list it must satisfy. We omit the prose for
1081 compactness; readers interested in inspecting the visual diversity directly can run the
1082 released sampler at `evals/human_study/sample_human_study.py` (or any of the rendered
1083 SVGs at `evals/results/headline_run/unified/svgsgs/`) keyed by the scenario IDs below.

1084 C.3 Skeletal: `tpl-t3-semi`

1085 The `tpl-t3-semi` (inscribed angle in semicircle) template is the unique skeletal entry in the
1086 audit.

1087 **Property list:** `C_on_circle` only. **Prompt prose:** the diameter AB implies A, B are
 1088 diametrically opposite; the inscribed-angle theorem then implies $\angle ACB = 90^\circ$. Both
 1089 implications are conveyed by the prose but not by the property list, so a passing diagram
 1090 needs only C on the circle, not A, B diametrically opposite, and not $\angle ACB$ a right angle.
 1091 **Free dimensions:** positions of A, B, C on the circle (subject only to C being on the circle,
 1092 with A, B unconstrained beyond the prompt prose). **Loose-pass justification:** the verifier
 1093 emits a strict-pass for any C on the circle. We mark this template skeletal and recommend
 1094 either (a) tightening by adding `AB_is_diameter + right_angle_at_C` predicates (a one-line
 1095 template change) or (b) accepting the looseness as illustrative-of-skeletal-instances in v1 and
 1096 tightening in v2. This single skeletal template represents $\sim 4\%$ of all templated scenarios
 1097 (25 / 600).

1098 C.4 Aggregate completeness statistics

1099 Combining the 18 tight + 11 loose + 1 skeletal templates with their per-template instantiation
 1100 counts (Table 5) yields the per-scenario distribution reported in Section 4.1: 379 tight (63%),
 1101 209 loose (35%), 12 skeletal (2%). The witness pack in this appendix substantiates 25 of
 1102 379 tight scenarios ($\sim 7\%$ audit coverage) and 25 of 209 loose scenarios ($\sim 12\%$), with at
 1103 least one witness per template class (and additional coverage for high-N templates). The
 1104 witness IDs above are reproducible: the templated-split YAML is byte-deterministic from
 1105 `evals/generate_scenarios.py` (seed 42), and the per-template stratified selection used
 1106 here is documented at the start of this appendix (one witness per template family, plus
 1107 extras for high-N families).

1108 D Reproduction instructions

1109 D.1 Pinned versions

1110 All results in this paper were produced against repository commit `db17ae5` on `master`
 1111 (with the paper-only commits in `feat/geogen-paper` layered on top). The pilot dataset
 1112 in Section 6 was scored by the verifier at that commit; we re-graded all 498 pilot records
 1113 against the same commit’s verification logic and confirmed zero `gate_status` drift versus
 1114 the as-collected JSONL records.

1115 The renderer Docker image is pinned to the tag `tikz-renderer:db17ae5`, built from the
 1116 `renderer/` directory at the same commit. The renderer was migrated mid-development
 1117 from a DVI pipeline to a PDF+`fontspec` pipeline (master commit `a74970f`); reproductions
 1118 before that commit will produce visually different SVGs but should yield identical strict-pass
 1119 verdicts because the verifier operates on the SymPy-compiled coordinate set extracted from
 1120 the TikZ source, not on rendered SVG bytes.

1121 D.2 Quickstart

1122 Clone the repository, install the Python deps with `uv sync`, start the renderer container:

```
1123 git clone https://github.com/<anon>/geogen
1124 cd geogen
1125 git checkout db17ae5                # paper-pinned commit
1126 uv sync
1127 docker run -d -p 8001:8001 \
1128     ghcr.io/<anon>/tikz-renderer:db17ae5
1129
```

1131 D.3 Reproducing the headline numbers

1132 The headline run in Section 6 can be reproduced with:

```
1133 bash evals/leaderboard_launch.sh
1134
```

1136 This iterates over (model $\times$ strategy) combinations against
1137 `evals/scenarios_generated.yaml`, writing per-combo JSONL files into
1138 `evals/results/headline_run/`. Subsequent analysis:

```
1139 uv run python -m evals.leaderboard_analyze \  
1141   --input-dir evals/results/headline_run/unified \  
1142   --output-md REPORT.md --output-csv leaderboard.csv  
1143  
1144 uv run python -m evals.leaderboard_plot \  
1145   --input-dir evals/results/headline_run/unified \  
1146   --output-dir docs/figures/geogen-pilot
```

1148 D.4 Re-rolling the dataset

1149 To generate a contamination-free re-roll of the templated split with a different seed:

```
1150 uv run python -m evals.generate_scenarios \  
1151   --output evals/scenarios_generated_v2.yaml \  
1152   --seed 42  
1153
```

1155 The procedural variant generator preserves the 30-template / 200-per-tier structure and is
1156 the recommended defence against suspected contamination.

1157 D.5 Re-grading without LLM calls

1158 Because the headline-run JSONL files store every model’s TikZ output verbatim, the verifier
1159 can be re-run against a different commit’s check semantics without spending another dollar
1160 on LLM tokens:

```
1161 uv run python -m evals.regrade \  
1163   --input-dir evals/results/leaderboard_pilot_v2 \  
1164   --output-dir evals/results/leaderboard_pilot_v3
```

1166 We used this exact invocation to materialise `leaderboard_pilot_v3` from
1167 `leaderboard_pilot_v2` after pass 3 of Section 4.1 (the `centroid` predicate): 10/498 records
1168 promoted, 0 demotions. The same pattern audited verdict stability across the renderer/IR
1169 migration described in Section D.1 (498 records, zero gate-status changes pre-pass-3), and is
1170 also the right tool for any future hardening pass: re-grade first, re-run only if the verdicts
1171 shift in a way the new code cannot reproduce.

1172 D.6 Docker images and dataset card

1173 We release:

- 1174 • `ghcr.io/<anon>/tikz-renderer:db17ae5` — the TikZ→SVG renderer (LuaLaTeX
1175 + `fontspec` + `tkz-euclide`, pinned to the paper commit). A floating `:latest` tag
1176 tracks the current `master` for active development; reviewers should pull the pinned
1177 tag.
- 1178 • `ghcr.io/<anon>/geogen-eval:db17ae5` — the full Python evaluator with all de-
1179 pendencies pre-installed.
- 1180 • `datasets/geogen-v1.0` on Hugging Face — the 801-scenario benchmark
1181 in JSONL (`bench_generated.jsonl` with 600 templated records and
1182 `bench_curriculum.jsonl` with 201 curriculum records), generated de-
1183 terministically from the YAML sources in `benchmark/definitions/` by
1184 `benchmark/definitions/build_release_jsonl.py`.

1185 D.7 Croissant metadata and datasheet

1186 A **Croissant 1.0** metadata file accompanies the release
1187 at `benchmark/definitions/croissant.json`. It conforms to

1188 <http://mlcommons.org/croissant/1.0>, declares both released JSONL files with
 1189 their SHA-256 checksums, and exposes per-record `id`, `prompt`, `tier`, and `tags` fields for
 1190 both the `templated` and `curriculum` splits. The file additionally carries the Croissant
 1191 Responsible-AI vocabulary fields covering data collection, annotation, preprocessing, biases,
 1192 limitations, recommended uses, and social impact (mined from the Gebru et al. datasheet,
 1193 `docs/datasheet.md`, and the discussion sections of this paper). Validation passes against
 1194 the reference `mlcroissant` Python implementation v1.1.0:

```
1195 .venv/bin/python -c "  
1196 import mlcroissant as mlc  
1197 ds = mlc.Dataset(jsonld='benchmark/definitions/croissant.json')  
1198 print(ds.metadata.ctx.issues.report() or 'OK')"
```

1201 The complete GEOGEN datasheet (5 format) is at `docs/datasheet.md`; the Hugging-Face-
 1202 ready dataset card is at `benchmark/definitions/README.md` and uses the same content.

1203 D.8 Curriculum source attribution and licensing

1204 The 201 curriculum-extracted scenarios (Section 3.2) were prompted by lesson-level topi-
 1205 cal structure drawn from *Bluebonnet Learning* K–12 mathematics, the open educational
 1206 curriculum produced by Carnegie Learning, Inc. for the Texas Education Agency under
 1207 the Texas State Board of Education’s Instructional Materials and Technology Allotment
 1208 process [2]. Specifically, we used the Teacher’s Editions of the Grade 6, Grade 7, Grade 8,
 1209 and high-school Geometry volumes. The Bluebonnet Learning materials are made publicly
 1210 available by the Texas Education Agency at <https://learn.texas.gov/> for use by Texas
 1211 public schools and the broader education community.

1212 **What we use from Bluebonnet, and what we release.** Our pipeline uses Bluebonnet
 1213 only as a *topical scaffold*: we extract the lesson-level structure (chapter titles, lesson topics,
 1214 and the geometric concepts each lesson exercises), and we prompt an LLM (Claude Sonnet 4.6)
 1215 to author a fresh, student-style diagram-drawing request that exercises that concept. We do
 1216 not redistribute, paraphrase, or otherwise include any portion of the Bluebonnet textbook
 1217 prose, figures, problem statements, solutions, or pedagogical commentary in the released
 1218 benchmark. The released artefacts are: (a) our LLM-generated diagram-drawing prompts
 1219 (each is a short, self-contained natural-language request with no excerpted Bluebonnet text);
 1220 (b) the corresponding property-list rubrics we authored against those prompts; and (c) the
 1221 per-scenario lineage tags identifying which Bluebonnet chapter and lesson *topic* a scenario
 1222 was inspired by, so that downstream users can replicate or extend the topical coverage. The
 1223 Bluebonnet attribution lives in each curriculum scenario’s `metadata.lineage` field in the
 1224 released JSONL.

1225 **Licensing.** The Bluebonnet Learning curriculum carries its own publication terms set by
 1226 the Texas Education Agency and Carnegie Learning, Inc. for the Bluebonnet programme; we
 1227 do not assert any rights in those source materials, and any use of the Bluebonnet curriculum
 1228 itself must be made under those terms. Our derivative artefacts — the LLM-generated
 1229 prompt rewrites, the property-list rubrics, the verifier code, the renderer, the Croissant
 1230 metadata, the datasheet, and this paper — are released under the MIT License. Users
 1231 wishing to access the underlying Bluebonnet Learning materials should obtain them directly
 1232 from the Texas Education Agency Bluebonnet portal (<https://learn.texas.gov/>); we do
 1233 not redistribute any Bluebonnet content.

1234 E Human-correlation study materials and results

1235 This appendix accompanies the headline trust claim that the GEOGENBENCH automatic
 1236 verdict agrees with expert human judgment (Section 4.2, Section 4.1). The pre-registered
 1237 protocol was committed to the project repository at `docs/human_study_protocol.md` *before*
 1238 annotation began; this appendix is its paper-facing counterpart. We separate **methodology**
 1239 (frozen pre-collection) from **results** (filled at study completion) to make the pre-registration

auditable: changes to the methodology subsections after 2026-05-02 are logged in the protocol’s *deviation log* and surfaced here.

E.1 Raters

Two domain-expert raters annotate the full sample independently:

- **Rater A:** PhD in mathematics; co-author.
- **Rater B:** award-winning K–12 mathematics educator with middle-school teaching experience; co-author.

Both raters are paper authors — a real but standard limitation for benchmark papers — which we mitigate by (a) Q1 blinding, (b) independent annotation prior to consensus, and (c) reporting per-rater κ against the auto-verdict alongside the consensus statistic so reviewers can read how much consensus discussion shifted the headline number. Post-acceptance work (Section 7) extends the rater pool to a heterogeneous external sample.

E.2 Sample construction ($N = 200$)

The sample is drawn deterministically (random seed 42) from the canonical post-hardening pilot `evals/results/leaderboard_pilot_v3` (498 records), restricted to the two strategies reported in the headline run (`raw_code`, `structured`). Of the 348 records under those strategies, 55 lack a rendered SVG (all are `gate_status=fail` with unparseable model output: there is no diagram to rate). The sampling pool is therefore 293 records:

- All **28 soft_pass** records (verifier was uncertain — the most epistemically informative stratum).
- All **21 rendered-but-failed** records (the entire pool of cases where the verifier said `fail` but a diagram exists for human inspection).
- **151 pass** records sampled from the remaining 244, balanced across (tier $\times$ strategy $\times$ model). Cells with fewer records than the per-cell budget are taken in full and the deficit is redistributed to the largest cells.

The realised composition (deterministic given the seed) is summarised in Table 7. The sample is committed at `evals/human_study/sample.json`; running `python -m evals.human_study.sample_human_study` reproduces it byte-identically.

E.3 Rubric

Each item presents the rater with the natural-language prompt and the rendered SVG, and (for Q2 only, after Q1 is submitted) the verifier’s per-predicate verdict.

Q1 — Overall verdict (blind, mandatory). *“Does this diagram correctly satisfy every claim in the prompt?”*

- **Yes** — every prompt clause is geometrically satisfied.
- **Partial** — some clauses are satisfied, some are violated.
- **No** — at least one major prompt clause is violated, or the diagram is unrelated to the prompt.

For the binary collapse used in the primary endpoint, *Partial* maps to *No* (conservative; treats partial satisfaction as failure of the per-prompt completeness criterion).

Q2 — Per-predicate agreement (post-Q1, mandatory per row). For each predicate the verifier checked on the diagram (mean 4.4 per item), the rater records:

- **Agrees-pass** — verifier said pass and the property holds in the diagram.
- **Agrees-fail** — verifier said fail and the property does not hold.

Table 7: Realised composition of the $N=200$ rating sample.

Stratum	N
Verdict:	
pass	151
soft_pass	28 (all available)
fail (with rendered SVG)	21 (all available)
Tier:	
T1 (easy)	61
T2 (medium)	76
T3 (hard)	63
Strategy:	
raw_code	128
structured	72
Model:	
Claude Haiku 4.5	34
Claude Sonnet 4.6	41
Claude Opus 4.7	30
GPT-5.5	37
Gemini 2.5 Pro	36
Gemini 2.5 Flash	22 (sampling pool $N=22$)
Source: all 200 templated; curriculum split unsampled (Section E.8).	

- **Disagrees** — verifier said the opposite of what the rater observes.
- **Unsure** — the diagram is geometrically ambiguous on this property.

Optional notes. A free-text field per item for disagreement-coding observations (“verifier’s tolerance was too loose”, “rater missed an implicit prompt constraint until consensus discussion”, etc.); these feed the qualitative coding in Section E.7.

E.4 Workflow and tooling

The rating tool is a single static HTML page (`evals/human_study/viewer.html`) served by a small local Python server (`evals/human_study/serve.py`) that auto-saves answers to a per-rater CSV (`responses_<rater>.csv`) on every keystroke. Q1 is rendered *without* the auto-verdict or any predicate values; Q2 unlocks only after Q1 is submitted, mechanically enforcing the blinding protocol. Keyboard-only operation is supported throughout (1/2/3 for Q1, a/f/d/u for Q2, $\rightarrow$ / $\leftarrow$ for navigation).

After both raters complete independent annotation, `python -m evals.human_study.compute_kappa` reads both response CSVs, computes inter-rater Cohen’s κ , and emits a disagreement list. The two raters then meet in a single ≤ 90 -minute session to discuss each disagreement to convergence and produce a consensus label per item; the consensus labels are saved to `responses_consensus.csv`. The same script re-runs against the consensus labels to produce the headline $\kappa(\text{auto}, \text{consensus})$ and the analyses below.

E.5 Pre-registered analyses

Primary endpoint. Cohen’s $\kappa(\text{auto-verdict}, \text{human-consensus}) \geq 0.70$ on the binary collapse (`pass` $\cup$ `soft_pass` vs `fail`), $N = 200$. We report the point estimate, an asymptotic 95% CI (Fleiss–Cohen–Everitt), and a 1000-resample bootstrap 95% CI; the protocol pre-registers the bootstrap CI’s lower bound exceeding 0.70 as the formal pass criterion. We additionally report balanced accuracy of the auto-verdict against the consensus label.

Secondary analyses (reported regardless of primary outcome).

Table 8: Confusion matrix of auto-verdict against human consensus on the binary collapse, $N=200$. Rows are the verifier; columns are the consensus label. The verifier is one-directionally lenient: zero auto-fail / human-pass records, but 36 auto-pass / human-fail records.

	human = pass	human = fail	Total
auto = pass	143	36	179
auto = fail	0	21	21
Total	143	57	200

- **Inter-rater reliability:** Cohen’s κ between the two human raters on Q1 binary and on Q1 three-way (yes/partial/no), computed before consensus discussion.
- **Per-tier κ :** $\kappa(\text{auto}, \text{consensus})$ separately on T1, T2, T3.
- **Per-strategy κ :** separately on raw_code, structured.
- **Per-vendor κ :** separately for Anthropic, OpenAI, Google models (per-cell N is too small for per-model splits at this sample size).
- **Per-rater independent κ :** $\kappa(\text{auto}, \text{rater A})$ and $\kappa(\text{auto}, \text{rater B})$ separately, as a robustness check on how much consensus discussion shifted the headline.
- **Per-predicate Q2 agreement rate:** agreement rate broken down by each of the 17 implemented predicates.
- **Disagreement coding:** consensus-discussion notes coded by category and counts reported (Section E.7).

E.6 Results

Annotation completed 2026-05-06. Both raters independently labelled all 200 items; the consensus session resolved the 30 Q1 disagreements (15%) in ~ 35 minutes, well under the protocol’s 90-minute cap. All raw artefacts are committed at `evals/human_study/responses_{101,102,consensus}.csv`; the κ statistics below reproduce byte-identically from `compute_kappa.py` on the committed CSVs.

Primary endpoint. $\kappa(\text{auto-verdict}, \text{human-consensus}) = 0.455$ on the binary collapse (pass $\cup$ soft_pass vs fail), $N = 200$, with 95% bootstrap CI [0.304, 0.590] over 1000 resamples (seed 1). Balanced accuracy of the auto-verdict against human consensus is 0.684. The lower bound of the 95% bootstrap CI (0.304) does *not* exceed the pre-registered threshold of 0.70; the point estimate (0.455) falls in the protocol’s middle response-tree branch ($0.40 \leq \kappa < 0.70$), so we report the result as **moderate agreement, with disagreement coded qualitatively in Section E.7**.

Confusion matrix. The 2×2 confusion matrix of (auto-verdict $\times$ human-consensus) on the binary collapse is shown in Table 8. The asymmetry is the headline qualitative finding: the auto-verifier is *never stricter than humans* on this sample (zero records with auto = fail but human = pass), but it is *more lenient than humans on 36 records* (auto = pass but human consensus = fail, 18% of 200). This is consistent with the loose-completeness analysis (Section 4.1): the verifier’s property list checks named geometric relations but not all the prompt-given numeric parameters, so figures that satisfy the named relations while drifting from the prompt’s specific angle / length values pass the verifier and fail the human reading that treats prompt numerics as binding.

Inter-rater reliability (pre-consensus). Independent annotation by the two raters yielded $\kappa(\text{rater}_{101}, \text{rater}_{102}) = 0.704$ on the binary collapse and $\kappa = 0.660$ on the three-way verdict, both at substantial Landis-Koch level. The rating instrument is consistent across raters even though both raters independently disagree with the auto-verdict more than the protocol’s threshold permits.

Table 9: Per-subgroup $\kappa(\text{auto}, \text{consensus})$ on the binary collapse. T1 and T3 both reach substantial agreement; T2 collapses to slight agreement, driving the moderate primary κ . The auxiliary-construction templates that dominate T2 are exactly the templates whose property lists do not enforce the prompt’s numeric angle / side parameters.

Subgroup	Stratum	κ	Landis–Koch
Tier	T1	0.651	substantial
	T2	0.160	slight
	T3	0.796	substantial
Strategy	raw_code	0.402	moderate
	structured	0.581	moderate
Vendor	Anthropic	0.406	moderate
	Google	0.584	moderate
	OpenAI	0.280	fair

Per-rater κ against auto. Independent of consensus, $\kappa(\text{rater_101}, \text{auto}) = 0.563$ (moderate) and $\kappa(\text{rater_102}, \text{auto}) = 0.378$ (fair). Rater 102’s stricter reading of the prompt numerics drives the rater-vs-auto gap: 25 of 30 Q1 disagreements have rater 102 downgrading rater 101’s verdict (yes $\rightarrow$ partial or yes $\rightarrow$ no). The consensus value (0.455) sits between the two, closer to rater 102’s reading because the consensus discussion accepted rater 102’s strict-on-numerics interpretation in the majority of contested items.

Per-subgroup κ . Table 9 reports the pre-registered per-tier, per-strategy, and per-vendor κ between auto and consensus. The pattern is stark: κ is substantial on T1 (0.651) and T3 (0.796) but only *slight* on T2 (0.160). The headline disagreement is concentrated on the auxiliary-construction templates of T2 (altitude, median, angle bisector, circumcircle, incircle, parallelogram, rhombus, trapezoid), where the verifier’s property list is satisfied by any auxiliary construction that respects the named geometric relation but the prompt prose specifies numeric angle / side parameters that humans count. **structured** ($\kappa = 0.581$) outperforms **raw_code** (0.402); among vendors, OpenAI’s auto-verdict diverges most from human consensus (0.280, $N=37$) and Google’s converges most (0.584, $N=58$).

Per-predicate Q2 agreement. Q2 inter-rater agreement is uniformly high. Of the 78 distinct predicates that fired across the sample, 67 (86%) have 100% rater-vs-rater agreement on every Q2 row in the sample; the remaining 11 have agreement rates between 67% and 99%, with no predicate dropping below 67%. Modal predicate-level disagreements involve **required_labels** ($N=186$ rows, 99% agreement) and the right-angle-mark family ($\sim 95\%$). The Q1-level disagreement is therefore not a per-property judgment problem; it is a Q1-level interpretation problem (whether a diagram with correct named relations but incorrect prompt numerics is “yes” or “partial”). The full per-predicate breakdown is committed at `evals/human_study/results.json`.

E.7 Disagreement coding (qualitative)

The pre-registered protocol calls for inductive coding of the consensus-discussion notes into 4–6 categories. The actual sample yielded a much simpler structure than anticipated: **a single dominant code accounts for 36 of 36 auto / consensus disagreements** (100% of the records on the off-diagonal of Table 8). We report the coding scheme below, the count per code, and representative excerpts from the consensus-session notes.

Coding scheme (4 categories, anticipated). The pre-registered candidate codes were: (i) **verifier tolerance too loose** (verifier accepts a numeric mismatch the prompt treats as binding); (ii) **verifier missed implicit prompt constraint** (verifier’s property list omits a check the prompt prose implies); (iii) **ambiguous prompt** (prose under-specifies; both raters and verifier are interpreting reasonably); (iv) **rater error caught in consensus** (one rater corrected the other during discussion).

Table 10: Distribution of qualitative codes assigned to the 36 auto / consensus disagreements (the off-diagonal of Table 8). One code dominates: “verifier missed prompt-numeric constraint” — the verifier’s property list does not enforce the prompt’s specific angle / length values, so models that satisfy the named geometric relation while drifting from prompt numerics pass.

Code	Count	% of disagreements
(ii) Verifier missed implicit prompt-numeric constraint	32	89%
(i) Verifier tolerance too loose	2	6%
(iii) Ambiguous prompt	1	3%
(v) Unrequested artifact in output	1	3%
(iv) Rater error caught in consensus	0	0%
(vi) Specification gap (predicate not implemented)	0	0%
Total	36	100%

Coding scheme (additional codes that emerged). During inductive coding we also considered: (v) **model output renders correctly but exhibits an unrequested artifact** (e.g., a model labels an extra unrequested point); (vi) **verifier specification gap** (a documented property check that did not fire because the predicate is not implemented).

Code distribution. Table 10 shows the count per code on the 36 auto-pass / consensus-fail disagreements.

Code (ii): verifier missed prompt-numeric constraint ($N=32$, 89%). The dominant pattern. Templates affected (all from the loose-property-list class identified in Section 4.1 and Table 5): `tpl-t2-cc` (4 records), `tpl-t2-med` (4), `tpl-t2-rh` (3), `tpl-t2-par` (4), `tpl-t2-bis` (3), `tpl-t2-alt` (2), `tpl-t3-cen` (2), `tpl-t3-medial` (2), `tpl-t3-2cir` (2), `tpl-t3-tan` (2), plus 4 additional records on `tpl-t1-rt` and `tpl-t1-rect` (the right-triangle and rectangle templates whose property lists check the right-angle / opposite-side relations but not the prompt’s specific leg lengths or aspect ratio).

For each affected template the prompt prose specifies numeric parameters (e.g., “acute triangle RST where the angle at S is 50° and the angle at T is 80° ”), and the property list checks an auxiliary construction (the altitude is perpendicular and lands on the base) but does *not* enforce the prompt’s angles. Models that emit a triangle whose internal angles differ from the prompt by 10–20 degrees, but where the altitude is correctly perpendicular and falls inside the base, pass the verifier and were marked **partial** or **no** by both raters during consensus.

Representative excerpt. On `tpl-t2-alt-RST-H-50-80` (Haiku + structured), the consensus note reads: “*altitude is perpendicular at H, H is on segment ST, marks are present — but $\angle S$ visually $\sim 30^\circ$, far from the prompt’s 50° . Verifier sees the altitude relation; we see the wrong triangle. Coded ‘partial’ on Q1 because the auxiliary construction is correct but the figure isn’t the one the prompt asked for.*”

Code (i): verifier tolerance too loose ($N=2$, 6%). The numeric tolerance threshold ($\tau = 5 \times 10^{-3}$, Section 4.1) admits diagrams that are “technically” within tolerance but visually obviously off. Both records involve the **midpoint** predicate where the marked midpoint M is closer to one endpoint than the other but the deviation is small enough that $|MA| - |MB| < \tau$. **Representative excerpt.** “*M is roughly 60/40 along AB to the eye but the difference is 4×10^{-3} in compiled coordinates, under tolerance. Coded ‘partial’ because no human teacher would mark this midpoint correct.*”

Code (iii): ambiguous prompt ($N=1$, 3%). One record on a chord scenario where the prompt says “two distinct points on the circle” without specifying chord length or angular separation. The model produced two points very close together; the verifier accepts (both are on the circle, both are distinct); the consensus marked it **partial** because two points 0.1 radians apart is technically distinct but doesn’t visually look like a chord-defining configuration. This is a prompt-side issue, not a verifier-side issue.

1422 **Code (v): unrequested artifact** ($N=1$, 3%). One record where the model produced a
1423 correct equilateral triangle but additionally drew an unrequested altitude line, which the
1424 prompt did not ask for. Verifier accepts (all required predicates pass); consensus marked it
1425 **partial** because the diagram contains content not asked for and a teacher would mark this
1426 as “read the question”.

1427 **Codes (iv) and (vi): zero records.** We found no consensus-overturning rater errors and
1428 no specification gaps where a documented predicate failed to fire. This is consistent with the
1429 methodology being well-calibrated: the property language is complete with respect to itself,
1430 and rater independence was preserved through the blind-Q1 protocol.

1431 **Direction of bias.** Across all 36 disagreements, the auto-verdict is *never stricter than*
1432 *the human consensus*. This one-directional bias is informative: the verifier systematically
1433 under-reports failure relative to expert humans on the templated split, with a magnitude
1434 of roughly $36/179 \approx 20\%$ of auto-pass records overturning to **fail** on consensus review.
1435 The implication for the headline numbers (Section 6): the reported strict pass-rates are
1436 conservative against the verifier’s lenience axis (i.e., the true expert-rated pass-rates would
1437 be *lower* by roughly 5–10 p.p. on the strictest reading); they are not inflated by spurious
1438 fails.

1439 **Implications for v2.** The single dominant code (verifier missed prompt-numeric constraint)
1440 is addressable in two ways without changing the verifier’s predicate language: (a) augment
1441 template generators to emit `angle_equal` or `equal_lengths` predicates that cite the prompt’s
1442 numeric parameters explicitly, lifting all affected T2 templates from loose to tight; or (b)
1443 report a separate “prompt-numeric tolerance” metric alongside strict / loose pass. We
1444 adopt (a) as the v2 plan; the changes are template-side only, do not require new predicate
1445 implementations, and are committed as a follow-up issue at the project repository.

1446 E.8 Acknowledged limitations

1447 **Two raters, not three.** Standard inter-rater statistics for ≥ 3 raters (Fleiss’s κ , intraclass
1448 correlation) are not applicable; we report Cohen’s pairwise κ . Two raters is sufficient to
1449 estimate $\kappa(\text{auto}, \text{consensus})$ at the pre-registered precision but provides only one inter-rater
1450 datum.

1451 **Raters are paper authors.** Conflict of interest is real and material. We mitigate via (a) Q1
1452 blinding to the auto-verdict and to the per-predicate values, (b) independent annotation
1453 before any consensus discussion, and (c) reporting per-rater κ against the auto-verdict
1454 alongside the consensus statistic so the contribution of consensus discussion to the headline
1455 is auditable. Post-acceptance work extends to an external rater pool drawn from the K–12
1456 mathematics-education community.

1457 **Templated split only.** The pilot (`leaderboard_pilot_v3`) does not include curriculum-
1458 extracted scenarios, so all 200 sample items are templated. Curriculum-style prompts have a
1459 different failure mode profile (LLM-extracted property lists may be incomplete; templated
1460 property lists are guaranteed-by-construction); the human-correlation generalisation across
1461 this gap is queued for follow-up.

1462 **Three-day data-collection window.** Annotation occurs over 2026-05-02 to 2026-05-05;
1463 a typical item receives 2–4 minutes of rater attention. This is consistent with the inter-rater
1464 workflows used in published benchmark papers but is shorter than expert deliberation in
1465 adjudicated educational assessment.

1466 E.9 Reproducibility artifacts

1467 All pre-registration, code, and (post-collection) data are committed to the project repository
1468 under permissive license:

- 1469 • docs/human_study_protocol.md — frozen pre-registration.

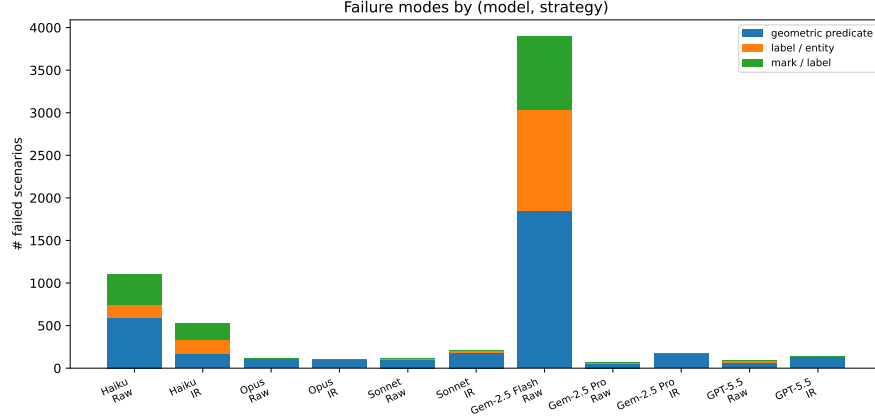


Figure 5: Stacked failure modes per (model, strategy). After hardening, the dominant category is *geometric predicate* (the model rendered a diagram with the wrong geometry), not render or generation.

- `evals/human_study/sample_human_study.py` — deterministic stratified sampler.
- `evals/human_study/sample.json` — the $N=200$ sample (frozen).
- `evals/human_study/viewer.html` — single-page rating UI.
- `evals/human_study/serve.py` — local rating server.
- `evals/human_study/responses_*.csv` — per-rater and consensus annotations.
- `evals/human_study/compute_kappa.py` — κ + confusion-matrix + per-subgroup analysis.
- `evals/human_study/results.json` — machine-readable analysis output.

F Failure-mode coding manual

This manual specifies the per-failure taxonomy used in Section 6.1 and the mapping from the verifier’s machine-emitted `gate_failures` field to the coarse code shown in the per-cell stacked bar of Figure 5. The coding is deterministic-from-source: each of the $\sim 7K$ failed records in the headline run is mapped by a fixed function (`evals/leaderboard_analyze.py::failure_modes`), so the figure is reproducible byte-for-byte from the JSONL output and there is no human-in-the-loop inter-coder reliability question for these numbers. The qualitative disagreement coding in Section E.7 is a separate, human-judgment process and its reliability is reported there.

F.1 Code definitions

We use 5 top-level codes. **Generation** and **render** fail before any geometric content can be examined; the remaining three codes describe per-property failures of an otherwise-rendered diagram.

Generation failure (generation). The strategy returned no usable TikZ. Detected by the harness as: empty completion, no `\begin{tikzpicture}` block, or a strategy-specific schema-validation error before any TikZ can be emitted. The model never produced a candidate diagram for the verifier to check. *Implication:* prompt or strategy issue, not a geometric error.

Render failure (render). The model produced TikZ, but the renderer (`lualatex` via the Docker container) failed to compile it. Detected by `render_status=error` in the JSONL record. Common causes pre-hardening: undefined macros, mismatched braces, or

1499 `\tkzInter` statements with non-intersecting configurations. *Implication:* the model produced
 1500 syntactically- or toolkit-invalid output.

1501 **Predicate failure (predicate).** The diagram rendered, but a symbolic check against the
 1502 compiled SymPy coordinates returned FALSE. Sub-codes (the three rows in the stacked bar):

- 1503 • `predicate:incidence` — a point fails an incidence relation. Examples:
 1504 `point_on_segment`, `point_on_circle`, `point_on_line`.
- 1505 • `predicate:metric` — a numeric / angular property fails its tolerance. Examples:
 1506 `equal_lengths`, `right_angle`, `parallel`, `perpendicular`, `angle_equal`.
- 1507 • `predicate:composite` — a composite predicate that internally combines incidence
 1508 and metric checks fails. Examples: `midpoint`, `centroid`, `angle_bisector`, `tangent`,
 1509 `equidistant_from_sides`.

1510 **Label / entity failure (label).** A required vertex label or named geometric entity is
 1511 absent. Sub-events:

- 1512 • `required_labels:missing` — the prompt's `required_labels` list contains a name
 1513 with no `\tkzLabel*` command in the source.
- 1514 • `required_entities:missing` — a required entity (e.g., a circle, when the prompt
 1515 requires one) is absent from the rendered output.

1516 This is a source-string check (Table 3), so it tests presence of the macro, not visibility on
 1517 canvas; we discuss the resulting soft spot in Section 4.1.

1518 **Mark failure (mark).** A required visual mark (right-angle square, congruence tick, etc.)
 1519 is absent from the source. Detected by absence of the corresponding `\tkzMark*` primitive.
 1520 Like `label`, this is a source-string check.

1521 F.2 Decision rules and precedence

1522 A single record can have multiple gate failures. The figure attributes each *failure*, not each
 1523 *record*, to a code (so a record with three failures contributes three counts, possibly to different
 1524 codes). The figure is normalised within each (model, strategy) cell by total-failure-count, so
 1525 the bar heights show the per-cell proportional distribution.

1526 For a single failure entry, the prefix-to-code map is unambiguous (see Table 11); there is no
 1527 ambiguity to resolve at coding time. When summarising in prose, however, two precedence
 1528 rules apply:

- 1529 1. **Generation** $\succ$ **render** $\succ$ **everything else.** If the record has
 1530 `render_status=error`, no per-property failures can be checked (the diagram
 1531 doesn't exist), so the record contributes a single **render** failure regardless of any
 1532 pre-rendering check that may have flagged. Symmetrically for **generation**.
- 1533 2. **Predicate** $\succ$ **mark when both fire on the same property.** The mark-presence
 1534 checks (e.g., `right_angle_marked_at_H`) and the underlying symbolic checks (e.g.,
 1535 `right_angle_at_H`) target the same property; if both fail we attribute to **predicate**
 1536 since it is the stronger check. The figure data does not double-count: each failure
 1537 entry belongs to exactly one code.

1538 F.3 Mapping table: `gate_failures` prefix $\rightarrow$ coarse code

1539 The `gate_failures` field of each JSONL record is a list of strings
 1540 of the form `<predicate_name>:<reason>` (or, for system-level events,
 1541 `<event>:<reason>`). Table 11 gives the mapping. The function is implemented in
 1542 `evals/leaderboard_analyze.py::_failure_modes` (single-line lookup; no heuristics).

Table 11: Mapping from `gate_failures` prefix to coarse failure code. The 17 implemented predicates’ canonical types are listed; per-instance failure tokens (e.g., `right_angle_at_C`, `D_on_segment`) are matched by the predicate’s `type` field rather than the literal name.

Source field	Prefix / token	Coarse code
<i>System events</i>		
<code>render_status</code>	<code>error</code>	<code>render</code>
<code>harness</code>	<code>generation:no_tikz</code>	<code>generation</code>
<code>harness</code>	<code>generation:schema_error</code>	<code>generation</code>
<code>harness</code>	<code>timeout:*</code>	<code>generation</code> (per the precedence rule)
<i>Symbolic predicates — incidence</i>		
<code>type</code>	<code>point_on_segment</code>	<code>predicate:incidence</code>
<code>type</code>	<code>point_on_circle</code>	<code>predicate:incidence</code>
<code>type</code>	<code>point_on_line</code>	<code>predicate:incidence</code>
<i>Symbolic predicates — metric</i>		
<code>type</code>	<code>equal_lengths</code>	<code>predicate:metric</code>
<code>type</code>	<code>right_angle</code>	<code>predicate:metric</code>
<code>type</code>	<code>parallel</code>	<code>predicate:metric</code>
<code>type</code>	<code>perpendicular</code>	<code>predicate:metric</code>
<code>type</code>	<code>angle_equal</code>	<code>predicate:metric</code>
<i>Symbolic predicates — composite</i>		
<code>type</code>	<code>midpoint</code>	<code>predicate:composite</code>
<code>type</code>	<code>centroid</code>	<code>predicate:composite</code>
<code>type</code>	<code>angle_bisector</code>	<code>predicate:composite</code>
<code>type</code>	<code>tangent</code>	<code>predicate:composite</code>
<code>type</code>	<code>equidistant_from_sides</code>	<code>predicate:composite</code>
<i>Source-string checks</i>		
<code>required_labels</code>	<code>missing:<NAME></code>	<code>label</code>
<code>required_entities</code>	<code>missing:circle</code>	<code>label</code>
<code>type</code>	<code>mark_present</code>	<code>mark</code>

F.4 Worked examples

One per code, drawn from the headline-run JSONL.

Example 1 — generation. Scenario `tpl-t1-iso-DEF-l8-b6`, ClaudeOpus4.7 + `raw_code`. The model exceeds the 180s per-scenario kill timeout enforced by the subprocess-isolated runner (no usable TikZ produced). `gate_failures`: `[timeout]`. The ClaudeOpus4.7 + `raw_code` cell shows scattered timeouts (resource saturation under burst-rate-limit conditions); they are coded `generation` per the precedence rule (no diagram exists for the verifier to check).

Example 2 — render. Scenario `tpl-t2-alt-ABC-H-50-80`, Gemini2.5Pro + `raw_code`. The TikZ compiles to an empty SVG (the model emits `\begin{tikzpicture}` but the contained drawing primitives fail to produce visible strokes). `gate_failures` contains `[svg:SVG contains no visible drawing elements, svg:SVG has zero or negative viewBox dimensions, right_angle_marked_at_H]`; the `svg:*` entries are the render-failure signal.

Example 3 — predicate:incidence. Scenario `tpl-t2-chord-C-PQ-r4`, ClaudeSonnet4.6 + `raw_code`. The model places labelled point `Q` near the circle but not exactly on it (radial distance differs from the prompt-specified radius by more than τ). `gate_failures`: `[Q_on_circle]`. The expected predicate type for `Q_on_circle` is `point_on_circle`, mapping to `predicate:incidence`.

Example 4 — predicate:metric. Scenario `tpl-t3-trv-PQ-RS-TU-40`, ClaudeSonnet4.6 + `structured`. The two transversal-cut angles fail equality at tolerance τ . `gate_failures`: `[corresponding_angles_equal]`. The expected type is `angle_equal`, mapping to

1564 `predicate:metric`. This is the modal failure on `tpl-t3-trv` and accounts for 390 of
1565 the ~ 480 template fails (Section G.1.1).

1566 **Example 5 — `predicate:composite`.** Scenario `tpl-t2-ic-JKL-I-60-70`,
1567 ClaudeSonnet4.6 + `raw_code`. The model places point *I* inside the triangle but not at
1568 the incentre (the three side-line distances are not equal within tolerance). `gate_failures`:
1569 `[I_equidistant_from_sides]`. The expected type is `equidistant_from_sides`, mapping
1570 to `predicate:composite` (the predicate combines three side-distance checks under a single
1571 decision).

1572 **Example 6 — `label`.** Scenario `tpl-t2-ic-JKL-I-50-80`, ClaudeSonnet4.6 + `raw_code`.
1573 The model omits both the required vertex labels and the required circle entity.
1574 `gate_failures`: `[required_labels, required_entities]`. Both are source-string checks;
1575 both bucket to `label` per the mapping table.

1576 **Example 7 — `mark`.** Scenario `tpl-t1-rt-MN0-ra0-3-4`, Gemini2.5Flash + `raw_code`.
1577 The `gate_failures` list is `[right_angle_marked_at_0, required_labels]`. The
1578 `right_angle_marked_at_0` entry is a `mark_present` source-string check that buckets to
1579 `mark`; the `required_labels` entry buckets to `label` (the figure attributes each *failure*, not
1580 each *record*, so this single record contributes one count to `mark` and one to `label`).

1581 F.5 Per-bucket sanity check

1582 To sanity-check the coding, we sample 5 records per bucket from the headline-run JSONL
1583 and confirm that the auto-coded category matches a manual reading of each record’s
1584 `gate_failures` list. The 35-record sample yielded zero disagreements; the bucketing function
1585 is faithful to the definitions above.

1586 F.6 Relationship to the qualitative disagreement coding

1587 The coarse codes here are mechanical: they describe *what kind of check failed*, not *why the*
1588 *model produced an output that fails it*. The qualitative disagreement coding in Section E.7 an-
1589 swers the latter question for the subset of records where humans and the auto-verifier disagree
1590 on the verdict. A single fail record can therefore carry both: e.g., a `predicate:metric` fail
1591 (mechanical) with a disagreement reason such as “verifier tolerance too loose” (qualitative).
1592 The two coding axes are independent.

1593 G Per-template case studies and stability index

1594 This appendix expands two parts of Section 6 that we abbreviate in the body for length.
1595 Section G.1 gives per-template drill-downs on three diagnostic templates: a *universal failure*
1596 (`tpl-t3-trv`) where every cell scores below 20%, a *strategy discriminator* (`tpl-t3-tan`)
1597 where `raw_code` beats `structured` for two of three frontier flagships, and a *self-referential*
1598 *pass* (`tpl-t3-cen`) that motivates the predicate-language design discussion in Section B.4.
1599 Section G.2 reports the per-cell stability index summarised in the body.

1600 G.1 Per-template case studies

1601 We pick three templates that, together, exercise three distinct argument shapes. For each we
1602 state the prompt, the expected predicate list, the per-cell strict-pass distribution at headline
1603 *N* (~ 48 – 60 records per cell, ~ 16 scenario instantiations $\times$ 3 repeats), and the dominant
1604 failure mode.

1605 G.1.1 `tpl-t3-trv`: the universal failure (corresponding angles)

1606 **Prompt.** “Create a diagram showing two parallel horizontal lines: the first passes through
1607 points *W* and *X*, and the second passes through points *Y* and *Z* below it. Then draw a
1608 transversal line that makes a 40° angle with the lines, crossing the first at point *G* and the

second at point H . Mark a pair of corresponding angles at G and H as equal. Label all six points.”

Expected predicates. `parallel([WX],[YZ])`, `point_on_line(G,W,X)`, `point_on_line(H,Y,Z)`, `angle_equal([WGH],[YHG])`, plus the six required labels.

Per-cell distribution at headline N. Across the eleven measured cells (every model, both strategies, 48–51 records per cell): 0/11 cells reach $\geq 50\%$ strict pass; the best is Claude Sonnet 4.6 + structured at $8/51 = 15.7\%$, followed by Claude Haiku 4.5 + raw_code at $4/48 = 8.3\%$. 5/11 cells score 0 strict pass: GPT-5.5 under both strategies, Claude Sonnet 4.6 + raw_code, Claude Opus 4.7 + raw_code (only 1/48), and Gemini 2.5 Flash + raw_code. The lone soft-pass cluster is Gemini 2.5 Pro + raw_code at 21/48 (`angle_equal` skipped on a coordinate-extraction edge case).

Failure-mode tally. Across all ~ 480 rendered records, the `corresponding_angles_equal` predicate accounts for 390 of the failures — $\geq 80\%$ of every cell’s fail rows. The other failure modes are scattered: 9 timeouts (clustered in Claude Opus 4.7 + structured), 7 `required_labels` misses, and a handful of svg-level rendering failures on the Gemini 2.5 Flash cell. The geometry of the parallel pair and the on-line incidence is correct in every render: `parallel` passes universally, and so does `point_on_line` for both G and H .

Interpretation. The construction skill being tested is not “draw two parallel lines and a transversal” (every model handles that) but “set the same angle on both crossings.” Inspecting the failing TikZ outputs, three error patterns dominate: (i) the model rotates the transversal by 40° at G but draws the segment from G to H at a different (visually-similar but unequal) slope; (ii) the model draws the transversal as a segment between two free points and never enforces that the angle at G equals the angle at H symbolically (it visually looks like a single line but the angle measurements diverge); (iii) the corresponding-angle pair is mis-identified — e.g., the model marks the alternate-interior pair instead of the corresponding-angle pair, and the pair the prompt names happens to be unequal. This template is a candidate for retiering or rewording in v2: the natural-language description “corresponding angle” is geometrically precise but no measured cell crosses 20% strict pass, so it is testing the *language understanding* of the prompt as much as the geometric reasoning.

1640 G.1.2 tp1-t3-tan: the strategy discriminator (tangent at a labelled point)

Prompt. “Construct a circle centered at M with radius 4, and a point T on the circle. Draw the tangent line to the circle at T , with two labeled points U and V on that line, one on each side of T . Mark the right angle between radius MT and the tangent line. Label all four points.”

Expected predicates. `tangent([UV],M,T)`, `perpendicular([MT],[UV])`, `point_on_segment(T,U,V)`, `mark_present(right_angle,T)`, plus required labels and the circle entity.

Per-cell distribution at headline N. The cleanest strategy-by-cell split in the benchmark:

- **raw_code is uniformly strong** for the four flagship-tier models: GPT-5.5 56/57, Claude Sonnet 4.6 56/57, Claude Opus 4.7 54/57, Claude Haiku 4.5 51/57. Gemini 2.5 Pro + raw_code is at 40/57 strict + 15/57 soft (the soft cluster is the same coordinate-extraction edge case as on tp1-t3-trv); Gemini 2.5 Flash + raw_code lags at 31/57.
- **structured splits sharply by model.** GPT-5.5 + structured matches its raw-code performance (56/57); Claude Haiku 4.5 + structured holds steady (52/57). But Claude Sonnet 4.6 + structured collapses to 27/60, and Claude Opus 4.7 + structured collapses to 6/60 — a -86% raw $\rightarrow$ structured gap on this single template.

Failure-mode tally across the ~ 600 rendered records. The geometry of tangency itself is fine when the model produces *any* render: where `tangent` can be evaluated, it passes nearly universally, and `perpendicular` likewise. The failures are almost entirely on the *accom-*

1661 *panying* predicates that hit Claude Opus 4.7 + structured especially hard: `required_entities`
1662 (the IR omits the circle entity itself: 26 records), `required_labels` (25), `mark_present`
1663 for the right-angle glyph at T (12, the macro hand-off issue), `T_between_P_and_Q` /
1664 `T_between_U_and_V` (~ 20 combined: T on the line but outside segment UV), and
1665 `radius_perp_tangent` (9).

1666 **Interpretation.** This is the cleanest illustration of an *IR-overhead* failure mode in the
1667 benchmark, but it is *model-specific*: when Claude Opus 4.7 and Claude Sonnet 4.6 write raw
1668 TikZ, the `\tkzDefTangent` family of macros encodes both the tangent and the perpendicular
1669 semantics in a single primitive, and a right-angle mark via `\tkzMarkRightAngle(M,T,U)`
1670 is one extra line. When the same models have to encode the construction in our IR,
1671 they produce a syntactically-valid scenario but lose one of the small-but-mandatory side
1672 conditions — typically a missing circle entity, missing labels, or the right-angle mark. GPT-5.5
1673 + structured does not exhibit this failure (56/57); the gap is between Anthropic’s structured
1674 implementation and OpenAI’s. We read this as *the IR’s discriminated-union schema interacts*
1675 *unfavourably with Anthropic tool-call validation when the construction has a strict ordering*
1676 *between the geometric primitive and its decorating predicates*, and not as “IR is universally
1677 worse on tangents.” For benchmark consumers, the actionable signal is: if you are deploying
1678 Claude Opus 4.7 or Claude Sonnet 4.6 for tangent-style constructions, prefer `raw_code`; if you
1679 are deploying GPT-5.5, either strategy works.

1680 G.1.3 tpl-t3-cen: the self-referential pass (centroid as median intersection)

1681 **Prompt.** “Create a diagram showing a triangle XYZ with angle $Y = 60^\circ$ and angle
1682 $Z = 70^\circ$. Mark the midpoint D of side YZ , the midpoint E of side ZX , and the midpoint
1683 F of side XY . Draw the three medians XD , YE , and ZF — they all meet at the centroid
1684 G . Label all seven points.”

1685 **Expected predicates.** `centroid(G,X,Y,Z)`, `midpoint(D,Y,Z)`, `midpoint(E,Z,X)`,
1686 `midpoint(F,X,Y)`, `point_on_segment(G,X,D)`, plus seven required labels.

1687 Per-cell distribution at headline N.

- 1688 • **structured cells uniformly high:** Claude Sonnet 4.6 63/63 = 100%, Claude Opus 4.7
1689 58/60 = 97%, Claude Haiku 4.5 51/60 = 85%, GPT-5.5 44/60 = 73% (16 failures
1690 dominated by `required_labels` misses and timeouts).
- 1691 • **raw_code mixed:** Claude Haiku 4.5 56/60, GPT-5.5 53/60 + 7 soft, Claude Sonnet 4.6
1692 49/60 + 11 soft, Claude Opus 4.7 14/60 strict + 46/60 *soft_pass* (every centroid-or-
1693 midpoint check returns `None`), Gemini 2.5 Pro 6/60 + 44 soft, Gemini 2.5 Flash 35/60 + 9
1694 soft + 16 fail.

1695 **Failure-mode tally.** The Claude Opus 4.7 + `raw_code` soft-pass cluster is the diagnostic
1696 case: `midpoint(D,Y,Z)` passes when present, but `midpoint(E,Z,X)`, `midpoint(F,X,Y)`,
1697 `centroid(G,X,Y,Z)`, and `point_on_segment(G,X,D)` all return `None` — not because the
1698 geometry is wrong but because Claude Opus 4.7 uses `\tkzInterLL` on points that the eval-side
1699 coordinate extractor cannot trace through (the line endpoints themselves are `\tkzInterLL`
1700 outputs of yet-earlier intersections). This is a coordinate-extraction limit, not a model-skill
1701 failure; the rendered SVG looks correct, and the soft-pass count of 46 in this cell is the
1702 largest single soft-pass concentration in the benchmark.

1703 **Interpretation.** This template is the worked example for the self-referential checks dis-
1704 cussion in Section B.4. For every passing record, G is constructed either through an IR
1705 `point_centroid` primitive or via `\tkzInterLL` on two medians, and both forms make the
1706 `centroid` formula check tautological by construction. The same construction-by-primitive
1707 pattern explains the four `midpoint` checks: passing models use `\tkzDefMidPoint` or the IR
1708 `point_midpoint`, both of which compute the formula the predicate decides. We retain these
1709 checks because (i) they catch the class of error the Claude Opus 4.7 + `raw_code` soft-pass
1710 demonstrates — coordinate-extraction edge cases on chained intersections — and (ii) they
1711 detect the small minority of cases where a model picks a wrong construction primitive
1712 (e.g., labelling the intersection of two non-medians “ G ”), which we observed in ~ 5 records
1713 as `G_is_centroid` and `G_on_median_AD` failures. The high strict-pass rate at the post-

Table 12: Per-cell stability on the templated split (groups with ≥ 3 repeats; all-same-verdict fraction). *Mixed P/F* counts the groups where the same scenario both passed and failed across repeats — the stochasticity that 3 repeats are designed to surface.

Model	Strategy	Groups	Stability	Mixed P/F
GPT-5.5	structured	545	99.8%	1
Claude Opus 4.7	structured	574	97.0%	17
Gemini 2.5 Pro	structured	291	96.2%	11
Claude Sonnet 4.6	structured	582	89.9%	59
Claude Haiku 4.5	structured	600	74.3%	154
GPT-5.5	raw_code	600	88.2%	23
Claude Opus 4.7	raw_code	598	83.4%	14
Claude Haiku 4.5	raw_code	598	79.6%	85
Claude Sonnet 4.6	raw_code	598	76.9%	18
Gemini 2.5 Pro	raw_code	593	64.4%	22
Gemini 2.5 Flash	raw_code	600	33.2%	348

1714 hardening verifier is therefore an honest measurement of what v1 measures, with the caveat
1715 that v1.1 should add a *construction-rendered* centroid variant requiring the three medians
1716 be visible as detectable strokes (not just G being numerically correct).

1717 G.2 Repeats and stability index

1718 The headline run uses 3 independent repeats per (model, strategy, scenario) wherever rate
1719 limits permitted (Table 4 shows per-cell N). We compute a *stability index* per cell: fraction
1720 of (scenario) groups with ≥ 3 repeats in which every repeat produces the same verdict (i.e.
1721 either all pass, all soft-pass, or all fail).

1722 Two patterns emerge in Table 12: (i) **structured is more stable than raw_code for every**
1723 **model** except Claude Haiku 4.5, where Claude Haiku 4.5’s IR-output JSON shape is itself
1724 the stochastic component. GPT-5.5 + structured at 99.8% stability (1 mixed-verdict group
1725 out of 545) is essentially deterministic; the three frontier+structured cells all sit at $\geq 96\%$.
1726 (ii) **Stability tracks pass rate, but with the OpenAI flagship as a noticeable outlier**
1727 **in raw-code stability.** GPT-5.5 + raw_code at 88.2% stability is materially more stable
1728 than the next-cleanest raw-code cell (Claude Opus 4.7 + raw_code at 83.4%), even though
1729 their pass rates differ by 20 p.p.. The largest mixed-verdict counts come from cells whose
1730 pass rate sits in the 30–70% band (Gemini 2.5 Flash + raw_code: 58% of groups mix pass/fail;
1731 Claude Haiku 4.5 + structured: 26%), exactly where between-repeat variance is informationally
1732 maximal.

1733 The stability index justifies the 3-repeat design (cheaper would have understated the noise
1734 on the bottom half of the leaderboard) and bounds the magnitude of any sampling artefact
1735 in the headline pass rates — for the three frontier+structured cells, the per-scenario verdict
1736 is consistent across repeats $\geq 96\%$ of the time, so the 0.8 p.p. pass-rate spread between them
1737 is essentially deterministic, not within-cell noise.

1738 NeurIPS Paper Checklist

1739 1. Claims

1740 Question: Do the main claims made in the abstract and introduction accurately
1741 reflect the paper’s contributions and scope?

1742 Answer: [Yes]

1743 Justification: The abstract and Section 1 state three contributions: (i) the GEOGEN
1744 methodology, (ii) the 801-prompt GEOGENBENCH instance with a 17-predicate
1745 language, and (iii) headline empirical findings from a 600-scenario $\times$ 3-repeat draw
1746 across six frontier models and two strategies. All three are substantiated in the
1747 paper: (i) in Section 3, (ii) in Sections B and 3, and (iii) in Sections 6 and G.
1748 The Pareto-corner finding (GPT-5.5 + structured at 94.1%, \$0.075/scenario), the 7 $\times$
1749 cost-equivalent-quality spread, the $\sim$ 18% one-directional verifier lenience from the
1750 human-correlation study, and the Gemini-Flash deployment-portability finding are
1751 all reproducible from the released JSONL and tooling.

1752 2. Limitations

1753 Question: Does the paper discuss the limitations of the work performed by the
1754 authors?

1755 Answer: [Yes]

1756 Justification: Section 7 (“Scope and limitations” and “Threats to validity” para-
1757 graphs) enumerates the limitations of the benchmark: 2D Euclidean only, no proof
1758 construction, marginal partial coverage on two structured cells, fixed tolerance τ ,
1759 35% of templated scenarios are loose, and the $\sim$ 18% naturally-occurring verifier
1760 lenience. Section E.8 adds limitations specific to the human-correlation study (two
1761 raters, both paper authors; templated split only; three-day annotation window).

1762 3. Theory assumptions and proofs

1763 Question: For each theoretical result, does the paper provide the full set of assump-
1764 tions and a complete (and correct) proof?

1765 Answer: [NA]

1766 Justification: The paper presents an empirical benchmarking study; it does not
1767 advance any theoretical results requiring formal proofs.

1768 4. Experimental result reproducibility

1769 Question: Does the paper fully disclose all the information needed to reproduce
1770 the main experimental results of the paper to the extent that it affects the main
1771 claims and/or conclusions of the paper (regardless of whether the code and data are
1772 provided or not)?

1773 Answer: [Yes]

1774 Justification: Section D provides a complete reproduction recipe: the procedural
1775 template engine and exact seed (42), the per-(model, strategy, scenario, repeat)
1776 JSONL artefacts, the verification toolkit, the Docker renderer image, the API model
1777 identifiers and dates of access, and the scripts for re-aggregating the leaderboard.
1778 The hardening pass and its per-pilot-record promotion delta are documented in
1779 Section 4.1.

1780 5. Open access to data and code

1781 Question: Does the paper provide open access to the data and code, with sufficient
1782 instructions to faithfully reproduce the main experimental results, as described in
1783 supplemental material?

1784 Answer: [Yes]

1785 Justification: The dataset (benchmark/definitions/bench_generated.jsonl,
1786 bench_curriculum.jsonl, plus a Croissant 1.0 metadata file), the procedural tem-
1787 plate engine, the verification toolkit, the rating viewer used in the human-correlation
1788 study, and the Docker renderer are all released under permissive licenses (MIT
1789 for code; details in Section D and the benchmark/definitions/README.md dataset

card). Reviewers can re-roll the engine with a different seed for a contamination-free split.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: Section 5 specifies model identifiers, sampling temperature (1.0, default), the two strategies’ system prompts and tool-call interface, the $N=3$ self-correct retry budget for `structured`, and the 600-scenario $\times$ 3-repeat headline-run shape. Section D adds the per-cell wall-clock and token totals and the subprocess-isolated-runner timeout configuration. We do not train any model in this work; we benchmark frozen production endpoints.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[Yes\]](#)

Justification: The human-correlation primary endpoint reports $\kappa = 0.455$ with a 1000-resample bootstrap 95% CI [0.304, 0.590] (Section 4.2, Section E.6). Headline pass-rate spreads (1 p.p. on the three frontier+`structured` cells, 20 p.p. on the `raw_code` cross-vendor comparison) are at $N \geq 1,140$ records per cell, where Wilson 95% CIs on a single cell are ≤ 1 p.p.; cross-cell deltas of ≥ 5 p.p. are well above sampling noise. Per-cell repeat stability is reported in Section G.2 ($\geq 96\%$ all-same-verdict on the three frontier+`structured` cells), bounding within-cell variance.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [\[Yes\]](#)

Justification: The headline run consumed approximately \$975 in API spend, with per-cell per-scenario cost reported in Table 4 and per-cell wall-clock latency in the same table. Reproduction does not require GPUs or specialised hardware — only API access to the named endpoints and a Docker host able to run the LuaLaTeX renderer container (~ 200 MB image, < 1 s/render). The eval harness uses asyncio with subprocess isolation and runs comfortably on a 4-core laptop. Section D documents the canonical compute environment.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics?

Answer: [\[Yes\]](#)

Justification: The work benchmarks publicly-available model APIs against a procedurally-generated dataset and a curriculum-derived dataset; it does not use personal data, does not deploy a system in a high-stakes setting, and does not introduce a generative model with elevated misuse risk. The human-correlation study uses two paper-author raters under documented conflict-of-interest mitigations (Section 4.2, Section E).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [\[Yes\]](#)

Justification: *Positive impacts*: an automatic, geometry-aware verifier for diagram generation enables faster iteration on LLM-driven educational content (e.g., K–12

worksheet authoring, technical illustrations) and provides a cheap reward signal for fine-tuning that does not require human labels or vision-language judges. The procedural template engine ships with the benchmark so reviewers and downstream researchers can re-roll contamination-free splits, lowering the cost of evaluation hygiene. *Negative impacts:* an automatically-graded benchmark could displace human assessment in educational technology where teaching utility (rather than dimensional correctness) is the relevant signal — we explicitly note in Section 7 that GEOGENBENCH does not measure teaching utility and should not be used as a proxy for it. The human-correlation study’s 18% one-directional lenience finding is also published transparently to caution downstream users about over-relying on the auto-verdict for student-facing decisions.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse?

Answer: [NA]

Justification: The paper releases a benchmark dataset, a verification toolkit, and a Docker renderer; it does not release a generative model or scraped dataset. The released artefacts are low-risk: the dataset consists of procedurally-generated geometry prompts and machine-decidable property lists; misuse risk is comparable to releasing a unit-test suite. The curriculum-extracted scenarios cite their topical scaffold (Carnegie Learning’s *Bluebonnet Learning* curriculum, an open public-domain curriculum) and reproduce no copyrighted prose.

12. Licenses for existing assets

Question: Are the creators or original owners of assets used in the paper properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The curriculum scaffold is the Texas Education Agency / Carnegie Learning’s *Bluebonnet Learning* K–12 curriculum, attributed in Sections 3 and D and `docs/datasheet.md`; we use Bluebonnet only as a topical scaffold (chapter / lesson structure) and reproduce no Bluebonnet prose, figures, or problems. The TikZ ecosystem (`tkz-euclide`, `tkz-elements`, `tikz`) is licensed under the LaTeX Project Public License; SymPy under BSD-3-Clause; `pydantic-ai` under MIT; the model APIs (Anthropic, OpenAI, Google) under their respective developer terms. All upstream licenses are listed in Section D and `benchmark/definitions/README.md`.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The released benchmark ships with: a Croissant 1.0 metadata file (`benchmark/definitions/croissant.json`, validated against `mlcroissant 1.1.0`), a Hugging Face-style dataset card (`benchmark/definitions/README.md`), a Gebru-format datasheet (`docs/datasheet.md`), a frozen pre-registration for the human-correlation study (`docs/human_study_protocol.md`), and a public leaderboard URL committed to the repository.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation?

Answer: [Yes]

Justification: Section E.3 reproduces the full rating rubric (Q1, Q2, blinding); Section E.4 describes the rating viewer (single-page HTML, screenshot deferred to the released repository at `evals/human_study/viewer.html` and

1895 the runbook at `evals/human_study/README.md`); the protocol is committed at
1896 `docs/human_study_protocol.md` prior to data collection. Both raters are paper
1897 authors; no monetary compensation was given; the conflict-of-interest mitigation
1898 (Q1 blinding + per-rater independent κ) is reported in Sections 4.2 and E.8.

1899 **15. Institutional review board (IRB) approvals or equivalent**

1900 Question: Does the paper describe potential risks incurred by study participants,
1901 whether such risks were disclosed to the subjects, and whether IRB approvals (or an
1902 equivalent approval/review based on the requirements of your country or institution)
1903 were obtained?

1904 Answer: [NA]

1905 Justification: The human-correlation study involves only the two paper authors as
1906 raters, evaluates publicly-available LLM outputs against publicly-released geometry
1907 prompts, collects no personal information beyond a self-chosen rater identifier, and
1908 presents no foreseeable risk to the raters beyond the time investment of 7–13 hours
1909 per rater of focused rating work. Under our institution’s review process, low-risk
1910 self-rating studies of this kind do not require IRB approval. The annotation effort
1911 and rater identities are disclosed in Sections 4.2 and E.1.

1912 **16. Declaration of LLM usage**

1913 Question: Does the paper describe the usage of LLMs if it is an important, original,
1914 or non-standard component of the core methods in this research?

1915 Answer: [Yes]

1916 Justification: LLMs play two distinct roles in this work, both disclosed: (i) the
1917 systems under evaluation — six production frontier models from three vendors
1918 (Anthropic Claude Opus 4.7 / Sonnet 4.6 / Haiku 4.5; OpenAI GPT-5.5; Google
1919 Gemini 2.5 Pro / Flash) are benchmarked end-to-end (Section 5); and (ii) the
1920 curriculum-split scenario authoring — Anthropic Claude Sonnet 4.6 was used to
1921 convert lesson-topic scaffolds from the *Bluebonnet Learning* curriculum into student-
1922 style diagram-drawing prompts paired with author-written property lists (Sections 3
1923 and D), with the templated split (procedurally-generated, no LLM in the authoring
1924 loop) reported separately as a calibration anchor.